

LibreNotebook

Enoch Yu

Last Updated: August 14, 2026

Contents

Preface	2
Purpose and Motivation	3
Contribute	3
 I Olympiad Algebra	 4
1 Inequalities	5
1.1 Classic Inequalities	5
1.1.1 AM-GM Inequality	5
1.1.2 Power Mean and Weighted Power Mean Inequality	8
1.1.3 Cauchy-Schwarz Inequality	8
1.1.4 Hölder's Inequality	9
1.1.5 Rearrangement Inequality	10
1.2 Advanced Inequalities	11
1.3 Inequalities Techniques	11
1.4 Practice Problems	12
 II Machine Learning	 13
2 Fundamental Math for Machine Learning	15
2.1 Calculus	15
2.1.1 Foundations	15
2.2 Linear Algebra	18
2.2.1 Foundations	18
2.2.2 CONTINUE	21
2.3 Probability Theory	21
 3 Python Basics for Machine Learning	 24
3.1 Basic Syntax	24

3.2	Modules and Libraries	28
3.2.1	NumPy	28
3.2.2	Matplotlib	30
4	Foundations and Intuitions	32
4.1	Fundamental Terms and Concepts	32
4.1.1	ML Paradigms	33
4.2	Introduction to Large Language Models	34
4.2.1	Tokenization	35
4.2.2	Language Model Output	36
4.3	Improving Large Language Models	37
4.3.1	Introduction to Post-Training	37
4.3.2	Reasoning	38
5	Deep Dive into Deep Neural Networks	41
5.1	Definitions and High Level Overview	41
5.1.1	High Level Architecture	41
5.1.2	Training	43
5.1.3	Gradient Descent and Learning Rate	44
5.2	Conventions and Mathematical Interpretations	47
5.2.1	Deriving Equations and Backpropagation	48
5.3	Building Vanilla Neural Network From Scratch	51
5.3.1	Simple Stochastic Gradient Descent-Based Learning Engine	51
5.3.2	Vanilla Neural Network	65
6	Building Language Models	80
6.1	Bigram Language Model	80
6.2	Attention and Transformers	80
6.3	Building an LLM with PyTorch	81
7	AI and The Future	82
7.1	AI Safety	82
8	Further Resources	83

Preface

§ Purpose and Motivation

LibreNotebook initially started with personal notes. To be exact, it began with me studying for national math olympiad. Frankly, I was lost when studying for the national math olympiad as not many resources were available specifically for the high school division of our country's olympiad. Even if there are resources, it was either for middle school round or was commercial with high expense. Naturally, I began seeking for problems and resources from different countries. There, with open and public resources, I was able to learn and improve. Since then, I always wanted to return the blessings received or at least show my appreciation.

Over the 2026 summer break, I wrote notes on linear algebra and machine learning. With increasing lengths and details, I thought they could be more than just personal notes. With different handouts that I wrote for multiple purposes, I thought it would be possible to return the blessings received by doing the same of returning knowledge. LibreNotebook is the notebook that highlights understandings that were learned to be shared.

§ Contribute

Please visit [CONTRIBUTING.md](#) for more information.

Part I

Olympiad Algebra

Note 1

Inequalities

Inequality is perhaps the most amiable, but extremely in depth section to study in Algebra. It could range from simple inequalities such as solving for x in $x + 1 > 2$ to problems involving numerous advanced techniques and known results. As a result, although few inequalities in advanced problems could be proven by bashing, most would require an elegant solution. With that in mind, this chapter will begin with numerous classical inequalities such as the AM-GM inequality to advanced concepts such as majorization.

§1.1 Classic Inequalities

One of the first concepts that most students, including myself, study when getting started with olympiad is I believe inequalities. Many problems are solved by tweaking known inequalities and doing some clever substitutions. In this note, we will discuss fundamental and classic inequalities as an introduction to olympiad inequalities.

1.1.1 AM-GM Inequality

Let's start with AM-GM Inequality. I assume most of you know or at least have heard of this inequality as we learn it in middle and high school. AM-GM inequality stands for Arithmetic Mean-Geometric Mean inequality and literally compares those two means.

Definition 1.1.1: AM-GM Inequality

For $a_k \geq 0$, the following inequality holds with equality if and only if $a_1 = a_2 = \dots = a_n$.

$$\frac{\sum_{i=1}^n a_i}{n} \geq \sqrt[n]{\prod_{i=1}^n a_i}$$

You might be more familiar with $a + b \geq 2\sqrt{ab}$, but the inequality above is the generalized version.

Tip.

AM-GM as a fundamental inequality has lots of application. However in olympiads, it is used the most for cancellation and investigating the minimum.

Consider the following example.

Exercise 1.1.2

Prove the following inequality for $a, b, c, d \geq 0$.

$$(a + b)(b + c)(c + d)(d + a) \geq 16abcd$$

Solution.

Because a, b, c, d are non-negative, we could utilize AM-GM inequality. By AM-GM inequality, the following inequalities hold.

$$a + b \geq 2\sqrt{ab}$$

$$b + c \geq 2\sqrt{bc}$$

$$c + d \geq 2\sqrt{cd}$$

$$d + a \geq 2\sqrt{da}$$

Multiplying the inequalities,

$$(a + b)(b + c)(c + d)(d + a) \geq 2\sqrt{ab} \cdot 2\sqrt{bc} \cdot 2\sqrt{cd} \cdot 2\sqrt{da} = 16abcd$$

hold and $(a + b)(b + c)(c + d)(d + a) \geq 16abcd$.

Exercise 1.1.3**SECOND PROBLEM FOR AM-GM INEQUALITY**

When we discuss arithmetic and geometric mean, we normally include weighted means. Thus, it is natural to consider AM-GM inequality with weighted means.

Weighted AM-GM Inequality

Let's first get started with definitions.

Definition 1.1.4: Weighted AM-GM Inequality

For weights $w_1, w_2, \dots, w_n \geq 0$ such that $\sum_{i=1}^n w_i = w$ the following inequality holds.

$$\frac{\sum_{i=1}^n w_i a_i}{w} \geq \sqrt[w]{\prod_{i=1}^n a_i^{w_i}}$$

The equality holds if and only if $a_1 = a_2 = \dots = a_n$.

From the definition, we can notice that the original inequality is obtained with $w_1 = w_2 = \dots = w_n = \frac{1}{n}$. Here is a problem on weighted AM-GM inequality.

Exercise 1.1.5: 2020 IMO Problem 2

Show that for $a, b, c, d \in \mathbb{R}$ such that $a \geq b \geq c \geq d > 0$ and $a + b + c + d = 1$,

$$(a + 2b + 3c + 4d)a^a b^b c^c d^d < 1$$

holds.

([Video Solution](#))

Solution.

First, consider the following weighted AM-GM inequality for a, b, c, d with weights a, b, c, d respectively.

$$\frac{a \cdot a + b \cdot b + c \cdot c + d \cdot d}{a + b + c + d} \geq \sqrt[a+b+c+d]{a^a b^b c^c d^d}$$

$$a^2 + b^2 + c^2 + d^2 \geq a^a b^b c^c d^d$$

Therefore, it suffices to show that the following inequality holds.

$$(a + 2b + 3c + 4d)(a^2 + b^2 + c^2 + d^2) < 1$$

Consider inequalities below.

$$\begin{aligned} a^2(a + 2b + 3c + 4d) &< a^2(a + 3b + 3c + 3d) \quad (\because b \geq d) \\ b^2(a + 2b + 3c + 4d) &< b^2(3a + b + 3c + 3d) \quad (\because 2a \geq b + d) \\ c^2(a + 2b + 3c + 4d) &< c^2(3a + 3b + c + 3d) \quad (\because 2a + b \geq 2c + d) \\ d^2(a + 2b + 3c + 4d) &< d^2(3a + 3b + 3c + d) \quad (\because 2a + b \geq 3d) \end{aligned}$$

Integrating,

$$\begin{aligned} (a + 2b + 3c + 4d)(a^2 + b^2 + c^2 + d^2) &< a^2(a + 3b + 3c + 3d) + b^2(3a + b + 3c + 3d) \\ &\quad + c^2(3a + 3b + c + 3d) + d^2(3a + 3b + 3c + d) \\ &< (a + b + c + d)^3 = 1 \end{aligned}$$

and the inequality holds.

With the ideas of weighted AM-GM inequality in mind, we can further generalize our results for different means.

1.1.2 Power Mean and Weighted Power Mean Inequality

Power Mean inequality, also known as QM-AM-GM-HM inequality, compares the four major means with generalization of AM – GM inequality. Without simple notations, we can represent the Power Mean inequality as $M_2 \geq M_1 \geq M_0 \geq M_{-1}$.

Definition 1.1.6: Power Mean Inequality

For positive real numbers a_i , the inequality

$$\sqrt[n]{\frac{\sum_{i=1}^n a_i^2}{n}} \geq \frac{\sum_{i=1}^n a_i}{n} \geq \sqrt[n]{\prod_{i=1}^n a_i} \geq \frac{n}{\sum_{i=1}^n \frac{1}{a_i}}$$

holds with equality if and only if $a_1 = \dots = a_n$.

With the definition in mind, we can naturally apply the same for weighted power means.

Definition 1.1.7: Weighted Power Mean Inequality

For positive real numbers a_i , the inequality

$$\sqrt[n]{\frac{\sum_{i=1}^n w_i a_i^2}{w}} \geq \frac{\sum_{i=1}^n w_i a_i}{w} \geq \sqrt[n]{\prod_{i=1}^n a_i^{w_i}} \geq \frac{w}{\sum_{i=1}^n \frac{w_i}{a_i}}$$

holds with equality if and only if $a_1 = \dots = a_n$.

Continuing from AM-GM inequality and its variations, the second most taught inequality in middle and high schools is probably Cauchy-Schwarz Inequality.

1.1.3 Cauchy-Schwarz Inequality

As always, let's get started with the definition.

Definition 1.1.8: Cauchy-Schwarz Inequality

For all real numbers a_i and b_i ,

$$\left(\sum_{i=1}^n a_i^2 \right) \left(\sum_{i=1}^n b_i^2 \right) \geq \left(\sum_{i=1}^n a_i b_i \right)^2$$

with equality if and only if $\frac{a_i}{b_i} = k$ for all integer $1 \leq i \leq n$, where $k > 0$ and $a_i b_i \neq 0$.

When $n = 2$ we get the form $(a^2 + b^2)(x^2 + y^2) \geq (ax + by)^2$ that we initially learn. Continuing, we can obtain an interesting direct consequence. The lemma below known as Titu's lemma, T2 Lemma, Sedrakyan's inequality, and Engel's form and was named after one of the greatest mathematician Titu Andreescu.

Corollary 1.1.9: Titu's Lemma

For positive real numbers a_i and b_i ,

$$\sum_{i=1}^n \frac{a_i^2}{b_i} \geq \frac{(\sum_{i=1}^n a_i)^2}{\sum_{i=1}^n b_i}$$

Proof.

Substituting $x_i = \frac{a_i}{\sqrt{b_i}}$ and $y_i = \sqrt{b_i}$ to the Cauchy-Schwarz inequality, the following inequalities are obtained.

$$\begin{aligned} \left(\sum_{i=1}^n x_i^2 \right) \left(\sum_{i=1}^n y_i^2 \right) &\geq \left(\sum_{i=1}^n x_i y_i \right)^2 \\ \left(\sum_{i=1}^n \left(\frac{a_i}{\sqrt{b_i}} \right)^2 \right) \left(\sum_{i=1}^n (\sqrt{b_i})^2 \right) &\geq \left(\sum_{i=1}^n \frac{a_i}{\sqrt{b_i}} \cdot \sqrt{b_i} \right)^2 \end{aligned}$$

Recasting,

$$\left(\sum_{i=1}^n \frac{a_i^2}{b_i} \right) \left(\sum_{i=1}^n b_i \right) \geq \left(\sum_{i=1}^n a_i \right)^2$$

and

$$\sum_{i=1}^n \frac{a_i^2}{b_i} \geq \frac{(\sum_{i=1}^n a_i)^2}{\sum_{i=1}^n b_i}$$

is obtained. Thus, the lemma holds.

This lemma comes especially handy if an inequality involves fractions with square numbers. Now that we discussed a direct consequence of Cauchy-Schwarz inequality, we can consider its generalization.

1.1.4 Hölder's Inequality

Cauchy-Schwarz inequality is actually a special case of Hölder's inequality. Both inequalities can be used in multiple ways such as eliminating radicals and fractions. Below is the definition of Hölder's inequality.

Definition 1.1.10: Hölder's Inequality

Let $a_i, b_i, \dots, z_i > 0$ and $\lambda_a, \lambda_b, \dots, \lambda_z \geq 0$ such that $\lambda_a + \lambda_b + \dots + \lambda_z = 1$. Then,

$$\left(\sum_{i=1}^n a_i \right)^{\lambda_a} \left(\sum_{i=1}^n b_i \right)^{\lambda_b} \cdots \left(\sum_{i=1}^n z_i \right)^{\lambda_z} \geq \sum_{i=1}^n a_i^{\lambda_a} b_i^{\lambda_b} \cdots z_i^{\lambda_z}$$

holds with equality if $a_1 : a_2 : \dots : a_n \equiv b_1 : b_2 : \dots : b_n \equiv \dots \equiv z_1 : z_2 : \dots : z_n$.

Indeed, Cauchy-Schwarz inequality does resemble Hölder's Inequality! To derive Cauchy-

Scharz inequality from Hölder's, it suffices to show that a specific case of Hölder's inequality will lead to Cauchy-Schwarz inequality.

Let $\lambda_a = \lambda_b = \frac{1}{2}$. Substituting, the following inequalities hold.

$$\begin{aligned} \left(\sum_{i=1}^n a_i\right)^{\frac{1}{2}} \left(\sum_{i=1}^n b_i\right)^{\frac{1}{2}} \left(\sum_{i=1}^n c_i\right)^0 \cdots \left(\sum_{i=1}^n z_i\right)^0 &\geq \sum_{i=1}^n a_i^{\frac{1}{2}} b_i^{\frac{1}{2}} \cdot c_i^0 \cdots z_i^0 \\ \left(\sum_{i=1}^n a_i\right)^{\frac{1}{2}} \left(\sum_{i=1}^n b_i\right)^{\frac{1}{2}} &\geq \sum_{i=1}^n a_i^{\frac{1}{2}} b_i^{\frac{1}{2}} \end{aligned}$$

Let $\alpha_i = \sqrt{a_i}$ and $\beta_i = \sqrt{b_i}$. Substituting,

$$\left(\sum_{i=1}^n \alpha_i^2\right)^{\frac{1}{2}} \left(\sum_{i=1}^n \beta_i^2\right)^{\frac{1}{2}} \geq \sum_{i=1}^n \alpha_i \beta_i$$

and

$$\left(\sum_{i=1}^n \alpha_i^2\right) \left(\sum_{i=1}^n \beta_i^2\right) \geq \left(\sum_{i=1}^n \alpha_i \beta_i\right)^2$$

is obtained. This is Cauchy-Schwarz inequality! Below is a very classic problem on Hölder's Inequality

Exercise 1.1.11

Show that the following inequality holds for all $a, b, c > 0$.

$$(a^3 + 2)(b^3 + 2)(c^3 + 2) \geq (a + b + c)^3$$

([Video Solution](#))

Solution.

The key to this problem is changing the left-hand side to apply Hölder's Inequality. Consider the following inequality by Hölder.

$$(a^3 + 1 + 1)^{\frac{1}{3}} (1 + b^3 + 1)^{\frac{1}{3}} (1 + 1 + c^3)^{\frac{1}{3}} \geq a + b + c$$

Thus, cubing both sides leads to the desired result.

Continuing from Cauchy-Schwarz inequality and its variation, another fundamental inequality of different type is rearrangement inequality.

1.1.5 Rearrangement Inequality

Let's start with definition.

Definition 1.1.12: Rearrangement Inequality

Let a_i and b_i be real numbers such that $a_1 \geq a_2 \geq \cdots \geq a_n$ and $b_1 \geq b_2 \geq \cdots \geq b_n$. Moreover, let $b_{\sigma(i)}$ be the i^{th} term of the permutations of $b_1, \dots, b_n$. Then, the following inequalities hold.

$$\sum_{i=1}^n a_i b_i \geq \sum_{i=1}^n a_i b_{\sigma(i)} \geq \sum_{i=1}^n a_i b_{n-i+1}$$

The equality holds if at least one sequence is constant.

With the definition, we can obtain a special consequence.

Corollary 1.1.13: Chebyshev's Inequality

For real numbers $a_1 \geq a_2 \geq \cdots \geq a_n$ and $b_1 \geq b_2 \geq \cdots \geq b_n$,

$$n \left(\sum_{i=1}^n a_i b_i \right) \geq \left(\sum_{i=1}^n a_i \right) \left(\sum_{i=1}^n b_i \right) \geq n \left(\sum_{i=1}^n a_i b_{n-i+1} \right)$$

holds with the equality if at least one sequence is constant.

Proof.

Consider the following cases where $b_1 \geq b_2 \geq \cdots \geq b_n$ are cycled through in the rearrangement inequality.

$$\begin{aligned} \sum_{i=1}^n a_i b_i &\geq a_1 b_1 + a_2 b_2 + \cdots + a_n b_n \geq \sum_{i=1}^n a_i b_{n-i+1} \\ \sum_{i=1}^n a_i b_i &\geq a_1 b_2 + a_2 b_3 + \cdots + a_n b_1 \geq \sum_{i=1}^n a_i b_{n-i+1} \\ &\vdots \\ \sum_{i=1}^n a_i b_i &\geq a_1 b_n + a_2 b_1 + \cdots + a_n b_{n-1} \geq \sum_{i=1}^n a_i b_{n-i+1} \end{aligned}$$

Adding the inequalities,

$$n \left(\sum_{i=1}^n a_i b_i \right) \geq \left(\sum_{i=1}^n a_i \right) \left(\sum_{i=1}^n b_i \right) \geq n \left(\sum_{i=1}^n a_i b_{n-i+1} \right)$$

and the corollary holds.

§1.2 Advanced Inequalities

§1.3 Inequalities Techniques

§1.4 Practice Problems

Part II

Machine Learning

§What is AI and Why Should We Learn It?

Unless you've been living under a rock, you probably have heard of the term "AI" and "Machine Learning". Just as the name suggests Artificial Intelligence is any technology that mimics human intelligence. That being said, AI is a very, very broad term. Under AI, there is machine learning (ML), which quite literally means techniques, or machines, that can learn from data to better mimic human intelligence. Now, just as us humans learn from data with our brain, a subset of ML is Deep Learning (DL), which uses neural networks inspired from our brains. We could continue more, but let's save that for later since this is a brief introduction.

AI has been studied, and will continue to develop. In October 2015, a legendary match between an AI model AlphaGo developed by DeepMind and one of the greatest Go player Lee Sedol was conducted. During the game, AlphaGo demonstrated its creativity through numerous moves including Move 37. Ultimately, AlphaGo won the game with 4-1 victory. More information can be found in [AlphaGo](#).

Despite the long history, the modern AI boom in the public occurred from late 2022 when OpenAI released its GPT-3.5. Nevertheless, LLMs have significantly transformed the lives of countless people. For instance, as the concept of "vibe coding" emerged, the concept of "coding" became more accessible to the public. Moreover, job markets are transforming with the rise of AI. Now despite all "AI bubbles" concerns and anti-AI movements, it is somewhat self-evident that AI and the concept of AGI (Artificial General Intelligence) will not suddenly disappear in near future. Then, the real question is "do you want to become a consumer or producer?"

To briefly share why I began studying ML, the reason is quite simple. As a student who wishes to advance as a quantitative researcher, not learning ML is just not in the option. Moreover, learning to develop AI will widen the future opportunities even if I decided different path than quantitative finance. Although the ideas on an AI model developing another model is rising, it is imperative that a human engineer must superintend the architecture and development. Finally, it is fun to learn. I mean what is more intriguing than to learn about your favorite tool and how to contribute to its development?

Fundamental Math for Machine Learning

This note covers fundamental math necessary for machine learning. Please note that I won't nor have the capacity to include details on the concepts that are not strictly necessary to learn ML, which I believe are out of the scope of the notes. With that in mind, let's get started!

§2.1 Calculus

Ah yes, that one friend who's everywhere, where ever you go. Broadly speaking, calculus is one of the foundational math for machine learning. For instance, backpropagation, which we will discuss further in latter notes, is fundamentally based on the chain rule. There are actually a lot to cover on calculus and there are some advanced topics. Good news is that you probably don't need calculus concepts outside multivariable calculus (a.k.a. Calculus III) unless you are doing theoretical works, and they are out of my capacity. That being said, I will include most calculus concepts that you will need for general purpose, but concepts that are out of scope of these notes will not be included.

2.1.1 Foundations

I am assuming most of you have some background in calculus, so I will be skipping basic concepts from Calculus AB/BC (a.k.a Calculus I and II) as this section is about calculus for machine learning. Please don't worry if you don't as you probably won't need all concepts from the courses to understand the topics here. Now, let's start by building from terminologies.

Definition 2.1.1

The **limit** of a function is the value that the function approaches as the input gets arbitrarily close to a value.

Below are some examples.

$$\lim_{x \rightarrow 3} x^2 = 3^2 = 9, \quad \lim_{x \rightarrow \infty} \frac{1}{x} = 0, \quad \lim_{x \rightarrow 1} \frac{1}{x-1} \text{ DNE}$$

Definition 2.1.2

Derivative of a function respect to a variable is the measure of the rate of change of the output based on the change of the variable(s). **Gradient** is a vector of partial derivatives. **Differentiation** is the process of finding the derivative.

With equations, derivative and partial derivatives can be written as the following for function $f(x)$ and $f(x_1, \dots, x_n)$.

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$$

$$\frac{\partial}{\partial x_i} f(a) = \lim_{h \rightarrow 0} \frac{f(a_1, \dots, a_{i-1}, a_i + h, a_{i+1}, \dots, a_n) - f(a_1, \dots, a_n)}{h}$$

For gradient ∇f , read as “nabla f”, the mathematical expression follows.

$$\nabla f(a) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(a) \\ \vdots \\ \frac{\partial f}{\partial x_n}(a) \end{bmatrix}$$

For instance, the gradient of $f(x, y) = x^2 + 2y$ can be written as $\nabla f = \langle 2x, 2 \rangle$ and the gradient at $(1, 1)$ can be written as $\nabla f(1, 1) = \langle 2, 2 \rangle$.

Now that we discussed about derivatives, let's take a look at common rules that will appear for machine learning. Starting with the power rule, below is the list of common rules.

Theorem 2.1.3: Common Rules for Derivatives

1. $\frac{d}{dx} x^n = nx^{n-1}$
2. $\frac{d}{dx} e^x = e^x$
3. $\frac{d}{dx} \ln(x) = \frac{1}{x}$
4. $\frac{d}{dx} a = 0$
5. $\frac{d}{dx} cf(x) = cf'(x)$
6. $\frac{d}{dx} (f(x) \pm g(x)) = \frac{d}{dx} f(x) \pm \frac{d}{dx} g(x)$
7. $\frac{d}{dx} (f(x)g(x)) = f'(x)g(x) + f(x)g'(x)$
8. $\frac{d}{dx} \left(\frac{f(x)}{g(x)} \right) = \frac{f'(x)g(x) - f(x)g'(x)}{(g(x))^2}$
9. $\frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x} \left(\frac{\partial f}{\partial x} \right)$
10. $\frac{\partial^2 f}{\partial y \partial x} = \frac{\partial}{\partial y} \left(\frac{\partial f}{\partial x} \right)$

Let's skip the proofs for this one! In addition to derivative rules, one of the most primary use case of calculus in machine learning is the chain rule. Indeed, the neural network is

fundamentally based on the chain rule in calculus.

Theorem 2.1.4: The Chain Rule of Calculus

The derivative of a composite function $h(x) = f \circ g(x)$ can be represented as $h'(x) = f'(g(x)) \cdot g'(x)$. Alternatively, for $y = f(u)$ and $u = g(x)$, the following equation holds.

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Proof.

Consider the following equation by the definition of derivatives.

$$h'(x) = \lim_{n \rightarrow 0} \frac{h(x+n) - h(x)}{n}$$

Substituting $h(x) = f(g(x))$, the following equations are obtained.

$$\begin{aligned} h'(x) &= \lim_{n \rightarrow 0} \frac{f(g(x+n)) - f(g(x))}{n} \\ &= \lim_{n \rightarrow 0} \frac{f(g(x+n)) - f(g(x))}{g(x+n) - g(x)} \cdot \lim_{n \rightarrow 0} \frac{g(x+n) - g(x)}{n} \end{aligned}$$

Let $n' = g(x+n) - g(x)$. Assuming that g is continuous, as $n \rightarrow 0$, $n' \rightarrow 0$. Therefore, the following equation is obtained.

$$h'(x) = \lim_{n' \rightarrow 0} \frac{f(u+n') - f(u)}{n'} \cdot \lim_{n \rightarrow 0} \frac{g(x+n) - g(x)}{n} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Thus, the chain rule holds.

For a brief example, let's take a look at an example with sigmoid function.

Exercise 2.1.5: The Derivative of Sigmoid Function

Sigmoid function is defined as $\sigma(x) = \frac{1}{1+e^{-x}}$. Find the derivative of $\sigma(x)$.

Solution.

Consider the following process with the product and chain rule.

$$\begin{aligned} \sigma'(x) &= \frac{d}{dx} (1 + e^{-x})^{-1} = -(1 + e^{-x})^{-2} \cdot (-e^{-x}) \\ &= (1 + e^{-x})^{-2} \cdot e^{-x} = (1 + e^{-x})^{-1} \cdot \frac{e^{-x}}{1 + e^{-x}} \\ &= \sigma(x) (1 - \sigma(x)) \end{aligned}$$

Therefore, the derivative $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

In addition to the chain rule above, we could expand it for multivariable functions.

Theorem 2.1.6: The Chain Rule for One Independent Variable

For differentiable functions $x = a(t)$, $y = b(t)$, and $z = f(x, y)$, the following equation holds.

$$\frac{dz}{dt} = \frac{\partial z}{\partial x} \cdot \frac{dx}{dt} + \frac{\partial z}{\partial y} \cdot \frac{dy}{dt}$$

Similarly, we could expand it for cases where x and y are functions of two variables.

Theorem 2.1.7: The Chain Rule for Two Independent Variable

For differentiable functions $x = a(u, v)$, $y = b(u, v)$, and $z = f(x, y)$, the following equations hold.

$$\begin{aligned}\frac{\partial z}{\partial u} &= \frac{\partial z}{\partial x} \frac{\partial x}{\partial u} + \frac{\partial z}{\partial y} \frac{\partial y}{\partial u} \\ \frac{\partial z}{\partial v} &= \frac{\partial z}{\partial x} \frac{\partial x}{\partial v} + \frac{\partial z}{\partial y} \frac{\partial y}{\partial v}\end{aligned}$$

For the purpose of these notes, let's skip the proof for the two theorems above.

critical points Linear approximation, Taylor approx, tangent planes

Vector calc

Jacobian and Hessian Matrices Lagrange Multipliers Fourier Transformation

§2.2 Linear Algebra

Linear algebra is a foundational building block of ML as majority of the components are written in very high dimensional arrays and tensors. Without exaggeration, training and using a model can be considered as performing billions of linear algebra operations. Therefore, it is encouraged to get familiar with the notations as they will be used throughout the future notes.

2.2.1 Foundations

First, let's get started with definitions. What is a vector? You probably have learned vectors as a quantity with both magnitude and direction. The same definition applies in math and ML, but how we represent and perceive them differs.

Definition 2.2.1

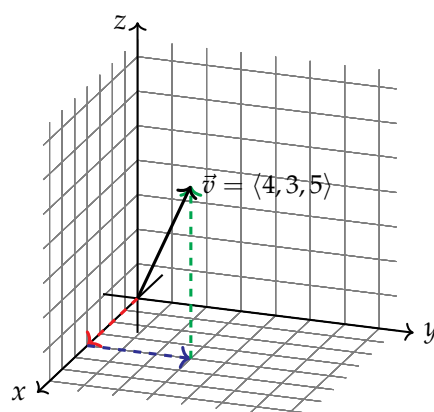
A **vector** is quantity with magnitude and directions. The set of vectors that satisfies addition and multiplication properties is called **vector space**. In math and ML, vectors are often represented as coordinates in vector space. **Dimensions** are the number of components that composes a vector.

Vectors with specific dimensions have dedicated names.

Definition 2.2.2

Scalars are 0-dimensional quantities. A 1-dimensional vector is simply called **vector**. A 2D vector is called **matrix**. For vectors with 3+ dimensions, the term **tensor** is used.

Now scalars are not considered vectors because it is only defined with magnitude, without direction. Geometrically speaking, the dimensions here can be interpreted as the dimension that we usually say in science. For example, a matrix can be plotted in a 2D space with x and y components. Similarly, a 3D tensor can be drawn in 3-dimensional plane with x , y , and z coordinates. Consider the following example.



Continuing, we can see different operations that vectors can perform.

Definition 2.2.3

Vector addition is a process of adding two vectors with same dimension elementwise.
Scalar multiplication is a process of multiplying a scalar to individual elements.

Let's take a look at quick examples. Consider the following equations.

$$\begin{aligned}\langle 1, 2, 3 \rangle + \langle 4, 5, 6 \rangle &= \langle 1 + 4, 2 + 5, 3 + 6 \rangle = \langle 5, 7, 9 \rangle \\ 2\langle 1, 2, 3 \rangle &= \langle 2 \cdot 1, 2 \cdot 2, 2 \cdot 3 \rangle = \langle 2, 4, 6 \rangle\end{aligned}$$

As you can see above, the operations are defined elementwise. For vectors of different dimension, we say their sum is undefined. Unlike addition and subtraction, multiplication is not defined elementwise. When considering "product" we consider dot product and matrix multiplication.

Definition 2.2.4

A **dot product** of two vectors is the sum of products of the corresponding elements.

Dot product will be quite important when we discuss attention and transformers because it can tell us how "related" two vectors are. Indeed, consider the vectors A and B with the angle

between the two vectors as θ . The following equation holds.

$$A \cdot B = \|A\| \|B\| \cos \theta$$

Here, $\|V\|$ for $V \in \mathbb{R}^n$ represents the magnitude of the vector, i.e. $\sqrt{\sum_{k=1}^n V_k^2}$. Let's take a look at a quick example.

$$\begin{aligned} \langle 1, 0 \rangle \cdot \langle 1, \sqrt{3} \rangle &= 1 \cdot 1 + 0 \cdot \sqrt{3} = 1 \\ &= 1 \cdot 2 \cdot \cos 60^\circ = 1 \end{aligned}$$

One thing to keep in mind is that we have a special term for the sign $\|V\|$.

Definition 2.2.5

The **vector norm** is the non-negative value that represents the magnitude of a vector.

With that in mind, let's discuss about matrices. There are so many different and interesting applications of matrices such as solving linear systems, but let's focus on the topics that will be essential in machine learning.

There are different ways of representing a matrix A , but a common way of writing is $[a_{ij}]$ where a_{ij} are the entries in i^{th} row and j^{th} column. A matrix $A_{m \times n}$ is said to have an **order** $m \times n$ if there are m rows and n columns. For example, below is a 3×2 matrix.

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

Similar to vector addition, matrix addition and scalar multiplications are defined elementwise. Moreover, the addition with matrices of different order is undefined. Consider the following equations.

$$\begin{aligned} [a_{ij}]_{m \times n} + [b_{ij}]_{m \times n} &= [a_{ij} + b_{ij}]_{m \times n} \\ \alpha [a_{ij}] &= [\alpha a_{ij}] \end{aligned}$$

Now matrix multiplication is very different from the operations above.

For matrices $[a_{ij}]_{m \times n}$ and $[b_{ij}]_{n \times p}$, the product matrix $[c_{ij}]_{m \times p}$ is defined as the following.

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

Below is an example.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 3 + 3 \cdot 5 & 1 \cdot 2 + 2 \cdot 4 + 3 \cdot 6 \\ 4 \cdot 1 + 5 \cdot 3 + 6 \cdot 5 & 4 \cdot 2 + 5 \cdot 4 + 6 \cdot 6 \end{bmatrix} \\ = \begin{bmatrix} 1 + 6 + 15 & 2 + 8 + 18 \\ 4 + 15 + 30 & 8 + 20 + 36 \end{bmatrix} = \begin{bmatrix} 22 & 28 \\ 49 & 64 \end{bmatrix}$$

Notice that the “inner numbers” of the orders must be the same for the multiplication to be defined. Moreover, if we multiply two matrices of orders $m \times n$ and $n \times p$ respectively, the multiplication is defined since $n = n$. The product matrix will now have the order $m \times p$. One more thing to note is that we don’t use the symbol $\cdot$ because that is reserved for dot products. We can interpret matrix multiplication in many ways, but let’s now move on to a more abstract and advanced topics in linear algebra for now.

2.2.2 CONTINUE

§2.3 Probability Theory

Ultimately, ML models are prediction engines. In other words, it is very crucial to understand fundamental probability theory. Therefore, let’s get started with basic terminologies and concepts.

Definition 2.3.1

Probability distribution is a process of describing probability and possible cases.

For example, using functions or tables to represents probability of getting a certain rolling sum after rolling a fair standard die two times is using probability distribution.

Few symbols to keep in mind are ω is used to represent the following of the probability distribution, and $\propto$ is used to illustrate direct proportionality.

Definition 2.3.2

Sample space, denoted as Ω , is a set of possible outcomes. **Probability measure** is a function that maps such outcome events to probability in $[0, 1]$. **Random Variables** are functions that map the outcomes to certain numbers.

Notice that the sum of all measures in a sample space is 1. Now sometimes, probabilities depend on the previous events.

Definition 2.3.3

If A is the event that occurs after B , the probability of A occurring after B is known as the **posterior probability**, denoted as $P(A | B)$. Moreover, $P(A)$ is the **prior probability**.

For instance, if A is the event where you see a snail outside and B is the event of raining, then the probability $P(A | B)$ is the probability of you seeing a snail outside given that it is raining. With this in mind, we can observe the following important theorem that will come very handy when we discuss language models.

Theorem 2.3.4: The Chain Rule of Probability

The chain rule of probability states the following equation.

$$P(A, B) = P(A | B)P(B)$$

The equation above is read as the probability of A and B is equal to the probability of A given B multiplied by the probability of B . The direct corollary of the chain rule is Bayes' Theorem.

Theorem 2.3.5: Bayes' Theorem

Conditional probabilities can be found with Bayes' Theorem defined with the equation below for $P(B) \neq 0$.

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

As you may have guessed, we can scale the chain rule above with generalization. Consider the following generalized version of the chain rule.

Theorem 2.3.6: Generalized Chain Rule

The chain rule of probability states the following.

$$\begin{aligned} P(x_1, x_2, \dots, x_n) &= P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2) \\ &\quad \dots P(x_n | x_1, x_2, \dots, x_{n-1}) \\ &= \prod_{i=1}^n P\left(x_i \mid \bigcap_{k=1}^{i-1} x_k\right) \end{aligned}$$

Proof.

Deriving this generalization from the chain rule with two variables is rather simple. We can use the definition multiple times to multiply and telescope. Here is what I mean. (Note that mathematicians like to use the standard set notations while ML engineers like to use a more "colloquial notation". For the purpose of these notes, let's use the notations that ML engineers use.)

Consider the following equations by the definition of conditional probability.

$$\begin{aligned}
 P(x_1) &= P(x_1) \\
 P(x_2 | x_1) &= \frac{P(x_1, x_2)}{P(x_1)} \\
 P(x_3 | x_1, x_2) &= \frac{P(x_1, x_2, x_3)}{P(x_1, x_2)} \\
 &\vdots \\
 P(x_n | x_1, \dots, x_{n-1}) &= \frac{P(x_1, \dots, x_n)}{P(x_1, \dots, x_{n-1})}
 \end{aligned}$$

Multiplying the equations, the following equation is obtained.

$$\begin{aligned}
 &P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2)P(x_n | x_1, \dots, x_{n-1}) \\
 &= P(x_1) \cdot \frac{P(x_1, x_2)}{P(x_1)} \cdot \frac{P(x_1, x_2, x_3)}{P(x_1, x_2)} \dots \frac{P(x_1, \dots, x_n)}{P(x_1, \dots, x_{n-1})}
 \end{aligned}$$

Telescoping the right hand side, we get our final equation.

$$P(x_1, \dots, x_n) = P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2)P(x_n | x_1, \dots, x_{n-1})$$

Alternatively, we could try going backward from the final equation from the definition, but I found this proof a little more satisfying.

Cross entropy

Note 3

Python Basics for Machine Learning

To develop a model represented in abstract math, we need a programming language. If you ask any ML engineer on the language they use the most, I can guarantee you that it's Python (well most likely). The reason is simple: syntax and libraries. Although all heavy computations in ML is mostly done by C++, Python is way easier to write than C++. You don't have to deal with segmentation faults and spend more time adding the missing semicolon than actually debugging. Moreover, Python has popular libraries like NumPy, Matplotlib, and PyTorch that will make our lives easier. Therefore, we will discuss about basic syntax, NumPy, and Matplotlib for ML in this note. Don't worry, we will discuss about PyTorch in latter notes as it requires ML concepts.

§3.1 Basic Syntax

As always, let's get started by being more familiar with the nature of Python. Unlike most other languages, indentation is a vital factor in Python. Two spaces, four spaces, doesn't really matter, let's use four spaces as it is more common and readable. Here is the Hello World example.

```
1 print("Hello World")
```

```
Hello World
```

Please note that this Jupyter Notebook can be found in `jupyter/` directory in GitHub under the root directory for these notes. Please feel free to download and follow along! The next example is a list of simple operations that we can do with Python.

```
1 import math
2 import numpy as np
3 import matplotlib.pyplot as plt
4 %matplotlib inline
```

```

5
6 print(1 + 2)      # Addition
7 print(3 - 9)      # Subtraction
8 print(4 * 3)      # Multiplication
9 print(5 ** 2)     # Power
10 print(math.log(5)) # log(a, base), default base is e
11 print(math.exp(2)) # e^2
12 print(math.e)     # Accessing constants

3
-6
12
25
1.6094379124341003
7.38905609893065
2.718281828459045

```

As you can see in Cell 2, we need module “math” to access constants and functions like e , π , and $\log_a b$.

Definition 3.1.1

We can achieve the same effect with **variables**, or a place with name that stores a value. A **function** is a reusable block of code dedicated for specific tasks.

Let’s take a look at an example of how function and variable in addition to the operations above can be used to find the approximation of the derivative of $f(x) = x^2 + 5$ at $x = 3$.

```

1 def f(x):
2     return x**2 + 5
3
4 a = 3
5 h = 0.0001
6 der = (f(a + h) - f(a))/h
7 print(der)

6.000100000012054

```

Let’s check if our answer is reasonable. Using the Power Rule, we know that $f'(x) = 2x$. Because $f(3) = 2 \cdot 3 = 6$ and 6.000100000012054 and 6 are somewhat close, our result is reasonable.

Definition 3.1.2

Lists are dynamic and ordered collection of elements of potentially different data types. An **index** is the position of an element in the list. The index of the first item is 0.

Below are examples of what we can do with lists.

```
1 # A list can be empty
2 a = []
3 print(a)
4
5 # We can append a value at the end
6 a = [1, 'hi', 3]
7 a.append('bye')
8 print(a)
9
10 # Remove an element
11 a.remove('hi')
12 print(a)
13
14 # Access an element with index
15 print(a[2])
16
17 # Find the number of elements
18 print(len(a))
```

```
[]
[1, 'hi', 3, 'bye']
[1, 3, 'bye']
bye
3
```

What do we do if we need to access multiple elements? This is where **slicing** shines. Consider the following example.

```
1 # a[i:j] to access from index 'i' to index 'j-1'
2 a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
3 print(a[1:5])
4
5 # Omit 'i' or 'j' to define only the start or end values
6 print(a[:6])
7 print(a[3:])
```

```
[2, 3, 4, 5]
[1, 2, 3, 4, 5, 6]
[4, 5, 6, 7, 8, 9]
```

Now that we know basic operations, lists, and how we can use functions, let's discuss about conditions and loops.

Definition 3.1.3

Conditional statements are statements that performs certain tasks under specific conditions. **Loops** are structure that repeats a block of code until they are explicitly told to end. One more definition that will make our lives easier is **increment**, which is an assignment operator that augments certain values.

Here is an example of conditional statements.

```
1 # If, Else, Elif (else if)
2 banana_num = 10
3 if banana_num < 10:
4     print("There are less than ten bananas")
5 elif banana_num == 10:
6     print("There are ten bananas")
7 else:
8     print("There are more than ten bananas")
```

There are ten bananas

Here, because “=” is reserved, we use “==” for “equal to.” We could also do less than or equal to, greater than or equal to, and not equal to with <=, >=, and != respectively.

Regarding loops, “for loop” and “while loop” are two primary methods. Below are examples of how both loops can be used.

```
1 # While loop
2 love_banana = True
3 banana_num = 0
4
5 while love_banana and banana_num < 5:
6     banana_num += 1 # Increment
7     print(banana_num)
8
9 print()
10
11 # For loop
12 my_banana = ['yellow', 'green', 'mushy', 'sweet']
13
14 for i in range(len(my_banana)):
15     print(my_banana[i])
```

1
2
3
4

```
5  
  
yellow  
green  
mushy  
sweet
```

First, `love_banana = True` is a Boolean value. In the while loop, what it does is while `love_banana` is `True`, we will increase the value `banana_num` by 1 and print them. Notice that the increment with `banana_num += 1` is a shorter form of writing `banana_num = banana_num + 1`. Moreover, we use `and` to make the loop continue if `love_banana` is `true` and `banana_num` is less than five.

In the second loop, we listed all elements in the list `my_banana`. The loop `for i in range()` tells Python to go through the code

```
print(my_banana[i])
```

 from $i = 0$ to $\text{range}(\text{len}(\text{my_banana})) - 1$.

We discussed very basic Python syntax that we must know to understand the following notes. Let's discuss core libraries NumPy and Matplotlib now, and save PyTorch for later.

§3.2 Modules and Libraries

First, what are modules and libraries?

Definition 3.2.1

Modules in Python are files that consists of definitions and statements for organization and reusability. **Libraries** are collection of modules and pre-written packages that can later be imported and distributed.

One of the reason why Python is so popular in ML and data science is because of the extensive libraries. Having the pre-written libraries specifically for ML and tensors like PyTorch simply makes our lives easier.

3.2.1 NumPy

NumPy is a Python package that helps handle multidimensional arrays. Let's get started by importing NumPy to our code. We could import it as whatever alias we want, but `np` is the standard for ML, so let's stick with that.

```
1 import numpy as np
```

Assuming that you have read the previous note or have strong foundations in linear algebra, we can implement the mathematical properties in Python with NumPy. Below is an example of

how we can define vectors in different dimensions. Note that `.ndim` can be used to find the dimension of a vector.

```
1 arr_1d = np.array([1, 2, 3])
2 arr_2d = np.array([[1, 2, 3],
3                   [4, 5, 6]])
4 arr_3d = np.array([[[1, 2], [3, 4]],
5                   [[5, 6], [7, 8]],
6                   [[9, 10], [11, 12]]])
7
8 print(f"Dimension: {arr_1d.ndim}, Shape: {arr_1d.shape}")
9 print(f"Dimension: {arr_2d.ndim}, Shape: {arr_2d.shape}")
10 print(f"Dimension: {arr_3d.ndim}, Shape: {arr_3d.shape}")

Dimension: 1, Shape: (3,)
Dimension: 2, Shape: (2, 3)
Dimension: 3, Shape: (3, 2, 2)
```

A simple way to understand multidimensional array in NumPy is stacking previous layers. For instance, `arr_2d` has the shape (2,3). We can think of it as stacking two 1D array with three elements. Similarly, `arr_3d` has the shape (3,2,2). We could think of it as stacking three layers of 2D arrays that consists of two 1D arrays. It is quite hard to visualize higher dimensional arrays, so let's take a look at different things that NumPy can do.

Below is an example where we utilize `arange`, `reshape`, `dtype`, and slicing techniques in NumPy. Let's create an array first.

```
1 # Making an array with numbers 0-17 and
2 # rearranging it to shape (3, 3, 2)
3 a = np.arange(18).reshape(3, 3, 2)
4 a

array([[[ 0,  1],
        [ 2,  3],
        [ 4,  5]],

       [[ 6,  7],
        [ 8,  9],
        [10, 11]],

       [[12, 13],
        [14, 15],
        [16, 17]]])
```

We can also find the data type with the attribute `.dtype`.

```
1 a.dtype.name  
  
'int64'
```

This means that the individual elements in array `a` are 8 bytes integers. Just as we index and slice 1D array, we could do the same with NumPy. Consider the following example for a matrix.

```
1 a = np.arange(9).reshape(3, 3)  
2 print(f"{a}\n")  
3  
4 # First row, second column element  
5 print(a[0, 1])  
6 # Entire second row  
7 print(a[1, :])  
8 # Entire third column  
9 print(a[:, 2])  
  
[[0 1 2]  
 [3 4 5]  
 [6 7 8]]  
  
1  
[3 4 5]  
[2 5 8]
```

This was the basic introduction to NumPy. Please check out the [documentation](#) for further information.

3.2.2 Matplotlib

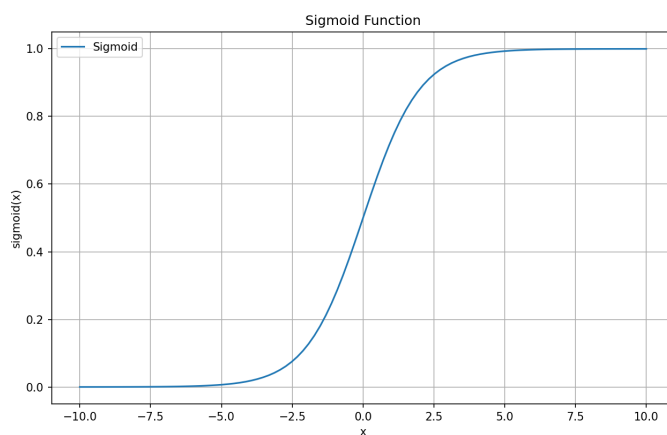
Do you want to visualize something with Python? Matplotlib is here for your need! Matplotlib is a Python library for visualization. Let's first import the library to take a look at an example.

```
1 import matplotlib.pyplot as plt  
2 %matplotlib inline
```

If you are using Jupyter Notebook like the examples in these notes, it is highly recommended to keep `%matplotlib inline` to make our lives easier by plotting directly in the notebook without an external window. Although Matplotlib is an essential library for visualizing model performance and analyzing, let's stick with basic plotting for now as I believe more information would be an overkill for this note.

For this brief introduction, let's plot Sigmoid function, which we will discuss in later notes.

```
1 # Define domain and the function
2 x = np.linspace(-10, 10, 100)
3 y = 1 / (1 + np.exp(-x))
4
5 plt.figure()
6 plt.plot(x, y, label='Sigmoid')
7 plt.grid(True)
8 plt.xlabel('x')
9 plt.ylabel('sigmoid(x)')
10 plt.title('Sigmoid Function')
11 plt.legend()
12 plt.show()
```



I know this intro was quite brief, but we will continue using amazing features in Matplotlib! For more information, please visit the official [tutorial page](#).

In the next note, we will discuss foundational terms and concepts in ML. It will also include a high level introduction to large language model (LLM). Understanding the high level overview of how LLMs operate under the hood really helped me grasp the intuition. Without further ado, let's get started!

Note 4

Foundations and Intuitions

This note covers basic concepts as well as intuitions through the introduction of Large Language Models.

§4.1 Fundamental Terms and Concepts

This section is an introduction to basic terms and concepts that will be used throughout my notes. That being said, more advanced concepts will be introduced in latter notes. Please feel free to skip over this section if you already know the bold words.

Definition 4.1.1

Machine learning is a subset of artificial intelligence that learns from data. **Deep learning** is a subset of machine learning that utilizes neural networks.

The primary purpose of ML is to make AI “smarter”. For instance, calculators are “dumb” in a sense that it requires an input and can only do certain tasks such as outputting calculated input. However, we can make it “smarter” by teaching data. We can clearly see that WolframAlpha is smarter than a normal scientific calculator. (Please note that this is just an analogy not an example because WolframAlpha is a knowledge engine, not a machine learning model. But I mean you get the point.)

On the high level, you can think of an AI model as a giant function with billions of array of numbers, called parameters.

Definition 4.1.2

Parameters are the internal values in a model composed with **weights** and **biases** that are learned by the model. After the parameters are finalized with the training phase, the models generate **predictions** through the process called **inference**.

Indeed, models are prediction machines based on the trained parameters. One fun fact here

is that it generally costs way less to run inference than to train the model. More in-depth explanation will be available in latter notes where we discuss neural nets.

4.1.1 ML Paradigms

In ML, there are three primary learning paradigms: supervised learning, unsupervised learning, and reinforcement learning. First, what does it mean to “learn” and “train”?

Definition 4.1.3

Machine learning utilizes the process of **training** to learn from data with algorithms. A common example would be dog and cat recognition from picture with **labeled** data.

There are multiple ways in which models are trained. One of the most used method out of the three paradigms is supervised learning.

Definition 4.1.4

When a model is trained with **supervised learning**, the model utilizes labeled dataset, or examples of input-output pairs, to learn.

With supervised learning, there are two primary predictions that models make.

Definition 4.1.5

Classification is a prediction where the label of an input data is being predicted. **Regression** is utilized to predict continuous values.

Classic examples of classification would be categorizing dog/cat pictures and email spam filtering. An example of regression would be the house price over years.

The second most common learning paradigm is unsupervised learning.

Definition 4.1.6

Unsupervised learning is a process of training a model with unlabeled data.

How is it possible for a model to learn without labeled data? One primary example algorithm is k -means clustering, where the model categorizes the data with relevance and similarity. To cluster high dimensional data, we often perform **dimensionality reduction** to reduce the dimension for clustering with the trust in **manifold hypothesis**. I won't go to deep into unsupervised learning and k -means clustering in this note, but you can check out this [awesome visualization](#) of k -means clustering.

The third most used learning paradigm is reinforcement learning.

Definition 4.1.7

Reinforcement learning is a process of training models through interactions with environment and reward system.

Personally, reinforcement learning is more complicated compared to the two paradigms introduced above. On the high level, an agent will take an action in an environment, where it returns **rewards** based on the action. The agent will then update its policies with the provided reward. The process is continued until the “adequate” amount is reached.

Training a model can be understood as writing a best fit mathematical function to training datasets that captures the holistic trend. There are few terms that describe the function.

Definition 4.1.8

A model is **overfit** if it learns too much from the data that it does not effectively capture that holistic pattern but small **noises**. On the other hand, an **underfitting** model is formed when it does not learn effectively from the data.

On the high level, our goal is to generalize and capture the general pattern and prevent overfitting or underfitting. We normally would stop training a model when it does not appear to improve, or is degrading, but how do we assess it?

Definition 4.1.9

To prevent overfitting and to evaluate the model during training, **validation** is utilized with **unseen data**. Unlike validation, **test sets** are used at the end of the training to evaluate the finalized model.

We have discussed the fundamental terms and concepts that would repeatedly appear in the following notes. Now let’s build our high level intuitions on ML with the introduction to LLMs!

§4.2 Introduction to Large Language Models

You probably have heard of Large Language Models (LLMs) and even have your favorite ones. You may have also noticed that if you and your friend ask a same question to a LLM, you get different response, or at least outputs that are not identical from word to word. That is because LLMs are generative models. As always, let’s get started by getting used to few vocabs.

Definition 4.2.1

Generative models are models that predicts and estimates the values on a probability distribution.

A generative model is also capable of taking in sequential data for a more context-rich prediction. An example of generative models would be an autoregressive model.

Definition 4.2.2

An **autoregressive model** is a generative model that predicts the future values in a sequence given the past input sequence.

For instance, when given the sequence of integers 0, 1, 1, 2, 3, 5, 8, an autoregressive model is likely to generate 13 by using previously learned pattern and the input. In other words, it is simply a model that predicts the term $P(x_n \mid x_1, \dots, x_{n-1})$.

Definition 4.2.3

A **language model** is a model that utilize human languages and predicts the next tokens given the previous inputs.

An interesting fact about such models is that they sometimes exhibit **emergent behavior** where they actually “learn” and exhibit capabilities that were not explicitly present in the training data.

4.2.1 Tokenization

We know that at the end, language models are huge machines that predict the next numerical values. Then how do such models generate outputs in human language? This is where **tokenization** kicks in.

Definition 4.2.4

Tokenization is a process of breaking down inputs into smaller units called **tokens** which are often represented as numeric values.

To tokenize inputs, a tool called **tokenizer** is used with **Byte Pair Encoding (BPE) algorithm**. Intuitively, what we could do is assign numbers to each alphabets like 1 for *a*, 2 for *b*, and so on. However, this would be very inefficient considering the frequency of each letter and scalability. Instead, what we can do with the BPE algorithm is create a byte-efficient list of merged chunks (that are not necessarily a single letter) based on frequency. Here is an example from the [OpenAI Tokenizer for GPT-5.x & O1/3](#).

Consider the following sentence:

Hippopotomonstrosesquippedaliophobia means fear of long words.

When tokenized, this 62 characters sentence turns into 17 tokens as following.

[‘H’, ‘ipp’, ‘opot’, ‘omon’, ‘st’, ‘ros’, ‘es’, ‘qu’, ‘ipped’, ‘ali’, ‘ophobia’,
‘ means’, ‘ fear’, ‘ of’, ‘ long’, ‘ words’, ‘.’]

The token IDs of the tokens above are the following numbers.

[39, 3012, 101543, 43960, 302, 2199, 268, 351, 8193, 3010, 141868,
4748, 11747, 328, 1701, 6391, 13]

Now, to help the models “understand” the tokens and take the IDs as input, we generally turn them into vectors through the process of **One-Hot Encoding (OHE)**, though the process may differ in modern Deep Learning for efficiency.

With OHE, the token ID i can be translated to a vector with 1 in index i and 0 in other slots. However, notice that OHE itself is not highly effective. To make the tokenizer more “meaningful” and “effective”, we could use the process of embeddings. **Learned embedding vectors** uses somewhat low dimensional vectors (usually a few thousand dimensions) to make the vectors “meaningful”. For example, in learned embedding vector, the difference between the vectors that represent king and queen will be similar to the difference between the vectors that represent man and woman. Modern models use **relative positional embeddings** to effectively distinguish the position of the words. (Let’s save this topic for latter notes on transformers.)

4.2.2 Language Model Output

As you probably have guessed, language models do not simply output the texts that we see. An autoregressive transformer language model will output a sequence of multi-dimensional vectors with probability distribution for the next token based on the sequence of multi-dimension input vectors.

Definition 4.2.5

Logits are the **scores** of the next possible token represented by individual vectors in the output sequence.

Note that logits are *raw* scores. To convert them to probability distribution, we use something called **softmax**, in which we will discuss further in notes about neural networks.

Definition 4.2.6

Decoding is the process selecting the next token to convert raw and abstract model output into a human language with **detokenization**.

There are different strategies in decoding, but the two main methods are **greedy decoding** and **sampling**. In greedy decoding, you select the token with the highest probability. Meaning, the temperature (we will cover this in neural nets notes) does not influence the selection. Unlike greedy decoding, temperature does influence the result from sampling. In sampling, you randomly select the next token with accounting the individual probability. To put it simply, imagine you have a biased die with thousands of sides. You roll the die with the bias in each side, and the next token is selected by the result.

Model outputs are random and highly organized. Take a look at the following example. In the first case, say the model input is “I love” and the expected model output is “I love potatoes”. In the second case, the model input is “Veni, vidi,” and the expected output is “Veni, vidi, vici”. When the model returns an output, which case do you think the model has the higher probability of returning the expected output? To answer this question, we must understand that models are composed of billions of parameters that are trained with countless data. The model likely learned lots of texts that includes “I love dogs”, “I love coffee”, and more. However, for a model dedicated for English, it is highly unlikely that it learned another phrase that starts with “Veni, vidi,” other than “Veni, vidi, vici”, the famous line from Julius Caesar. The model probably assigned lots of logits and converted them to probability distribution with softmax. While the first case have somewhat flat distribution, the second case will likely assign “vici” with high probability like 0.98 or 0.99. Therefore, the second model has the higher probability of returning the expected output.

Now that we have some intuitions on model outputs, let’s discuss about post-training.

§4.3 Improving Large Language Models

Up until now, we have discussed fundamental terms and very high-level overview of pre-training phase. This section discusses very high-level introduction to how a raw autocomplete model transitions to a model that we use in daily life.

4.3.1 Introduction to Post-Training

One of the most important phase of training that makes the chatbots chatbots are post-training. When a base model is developed through pre-training phase, it is simply an autocomplete model. Whereas, an aligned model is capable of generating conversation-like responses. For instance, if the input is “Why are dogs so cute?”, the base model will generate responses like “What are cats doing?”, which does not feel like a conversation. Whereas, an aligned model will generate responses like “People find dogs cute because humans developed a biological phenomena called baby schema.”

The post-training process contains two primary phases: Supervised Fine-Tuning (SFT) and Reinforcement Learning from Human Feedback (RLHF) On the high-level, SFT is the process where human provides prompt and response samples. The model then learns from the samples to provide a chatbot-like experience. After the phase, the model can be reinforced through RLHF. There are multiple ways of doing this, but a common method is making another “reward model” to judge and provide feedback on a model’s response. Instead of a human manually judging and rewarding every responses, reward model will predict how a human expert will reward (usually an integer from 0 to 5) a response to more effectively align the model. The specific process is quite out of scope for this note, and will be introduced in the following notes.

Most of the “learning” happens in pre-training. However, alignment is necessary to improve HHH, or helpfulness, honesty, and harmlessness. And yes, SFT and RLHF are popular, but they

are not the only methods. Few other examples contain [LIMA](#), [RLAIF](#), [DPO](#), [PPO](#), and [URIAL](#).

4.3.2 Reasoning

As you probably have noticed, LLMs are predictions models. In other words, they do not “reason” like how us humans reason. Language models will predict the next likely token that will provide a plausible response to a reasoning task. One way to test if the models can actually reason is by using benchmarks. One of the foundational datasets, which is considered outdated now, is [GSM8K](#) by OpenAI. This dataset is used to evaluate if a model can reason grade school level math. The modern, highly challenging, benchmark includes [FrontierMath](#). As you have noticed, benchmarking and reasoning is a giant hurdle for many LLMs. Modern engineers classify reasoning as another section in post-training to improve a model’s logic.

Prompting

One of the most effective way to improve LLM reasoning is to use prompting strategies.

Definition 4.3.1

One technique is **K-shot prompting** where you provide k examples for the model to follow a template. Zero-shot prompting will be when $k = 0$, where you provide no examples, but instructions. The second prompting strategy is [Chain-of-Thought \(CoT\) prompting](#). This strategy explicitly tells a model to break a complex reasoning tasks into intermediate steps.

Let’s take a look at an example on how K-shot prompting and CoT prompting work together.

Model Input

Instruction:

Consider the following examples to provide a response to the final equation. Provide a step by step explanation for your answer as demonstrated in the samples.

Example 1:

Q: Henry the dog loves tomatoes. He originally had 5 tomatoes. Henry ate 3 of them, and sneaked into the owner’s yard to secretly bring 4 more. How many tomatoes does Henry have now?

A: Henry started with 5, but ate 3. Therefore, he has $5 - 3 = 2$ tomatoes left. Because he brought 4 more from the owner’s yard, he now has $2 + 4 = 6$ tomatoes. Henry has 6 tomatoes now.

Example 2:

Q: Bob the yak loves burgers. One day, his lizard friend Sofia told him that she can bring

three boxes of burgers than contain 2 burgers each. Bob instantly pleaded Sofia, and he received the boxes. Out of hunger, he ate three burgers. How many burgers does he have now?

A: It is assumed that Bob does not have any burgers before the interaction with Sofia. With Sofia's virtue, Bob now has $2 \cdot 3 = 6$ burgers. Because he ate three burgers from six in total, Bob now has $6 - 3 = 3$ burgers left.

Question:

Charlotte the cat has five close friends. She wishes to split her 120 kg tuna equally with her friends. After the allocation, Charlotte ate 4 kg of tuna. How much tuna in kg does Charlotte have left?

Model Output

Charlotte initially has 120 kg of tuna. Because she split it with her friends, she has $\frac{120}{5+1} = 20$ kg of tuna after the allocation. Since Charlotte ate 4 kg of tuna from 20 kg, she now has $20 - 4 = 16$ kg of tuna.

As demonstrated, CoT has the potential to increase reasoning of LLMs. However, it is important to note that although performance may significantly ameliorate, numerous potential consequences follow. For instance, because the model includes the intermediate steps, the token generation considerably increases, which will increase the cost. Moreover, when model starts to hallucinate in the middle of the reasoning, the entire output from the hallucination will no longer be useful. Furthermore, models sometimes “lie”. What I mean to lie is that some models tend to provide a “plausible CoT” without actually providing one. This is more related to AI safety, and let's save that for final notes.

Self-Consistency

Another technique that could improve reasoning and performance is **self-consistency**.

Definition 4.3.2

Self-consistency is a technique where a model generates numerous possible outputs to an input, and aggregate the outputs for the final output.

The primary motivation of the technique is that just as mathematical problems have numerous paths to an answer, we could derive an “accurate” output with different reasoning. However, like any other techniques, there are limitations. One somewhat self-evident caveat is that this requires much greater computation. Because the technique requires multiple reasoning and aggregation for the final, the amount of computation drastically increase. One other issue is that this self-consistency only works on closed-solution prompts. I mean different people and different outputs can have differing opinions. Although this technique is not fully outdated, modern ML engineers use more effective and efficient techniques.

The field of reinforcement learning is one of the most rapidly developed field in ML. For instance, unlike RLHF, the emerging method Reinforcement Learning with Verifiable Rewards (RLVR) allows models to align themselves with minimum human contributions. In the next note, we will discuss about neural network from scratch!

Deep Dive into Deep Neural Networks

Neural networks (or neural nets) can be comprehended as a complex function that does not require manual feature engineering. Due to the high level of abstraction and automation in learning, neural nets are highly scalable. This note covers neural nets with mathematical interpretations and its application with Python.

§5.1 Definitions and High Level Overview

As always, let's get started with the fundamental terms and high level overview of what neural nets are. Before the introduction of neural nets, traditional models like linear/logistic regression and decision trees were used. The problem with traditional models is that you are likely required to manually adjust the **features**. With neural nets, the features are automatically engineered, making us to focus on the architecture.

5.1.1 High Level Architecture

To understand the high level architecture, consider the diagram below.

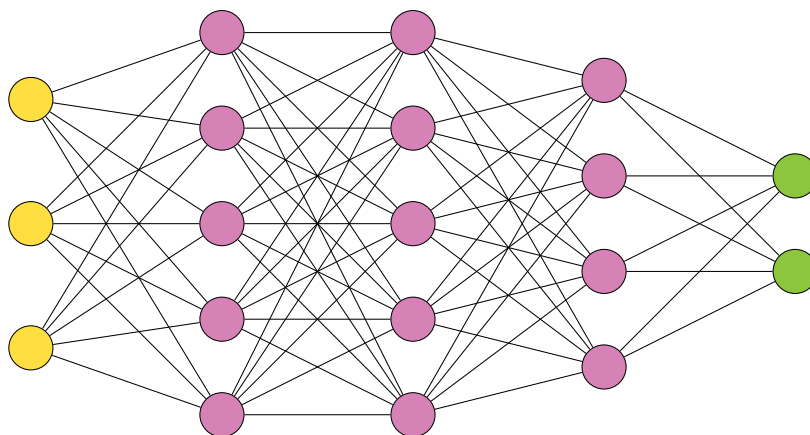


Figure 5.1: A visualization of a vanilla neural network (MLP)

From now, let's refer to the nodes as **neurons**.

Definition 5.1.1

Neurons are the fundamental unit in neural networks that process input into output with set of parameters.

From the diagram, the neurons are organized in to different columns. We call the columns **layers**.

Definition 5.1.2

The **input layer** is the layer that receives inputs for the neural network. **Hidden layers** are where the input data is processed with the learned parameters to extract information. The **output layer** is the final layer that represents the final prediction of the neural network.

In the diagram above, the input layer is represented as the layer with yellow neurons, hidden layers as the layers with purple neurons, output layer with the layer with green neurons. One of the advantages of neural network is that there is simply no limit, except computational limit. You can have unlimited amount of neurons in each layers and unlimited amount of hidden layers. The best neural net will be effectively structured in a way that there are sufficient neurons to learn from while avoiding overfitting and decreasing computational cost.

Definition 5.1.3

Connections are the links that connects the neurons with each other.

Just as our biological neurons are interconnected to each other, artificial neurons are connected to effectively extract the holistic feature from the input. Now it is worth noting that each neuron represents **feature**.

Here is a brief example. Say your neural net will classify handwritten numbers. The number 9 and 0 retains a similar feature. Both the numbers have a loop in the top. Therefore, in this scenario, the initial hidden layers may have similar features when categorizing the numbers.

In the past we used **perceptrons**, or neurons that uses step functions with binary output. However, modern multilayer perceptron (MLP) uses a more flexible neurons with continuous activation functions. Now let's take a look at what individual neurons have with the diagram below.

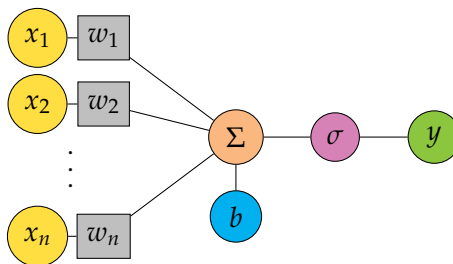


Figure 5.2: Visualization of a single neuron

On the high level, an individual neuron takes the weighted sum of the previous layer with the trained parameter. Bias is then added to make the activations more flexible. Then, an activation function is used to determine the final activation of the neuron.

Definition 5.1.4

Activation of a neuron indicates the significance of a neuron, or presence of a feature from the input data. An **activation function** is a function that generalizes each activation and introduce **non-linearity**.

In the diagram above, the purple color represents the activation function and the green color represents the final output of the neuron. Here, non-linearity is important for generalization with activation function. The activation is represented as a linear weighted sum. However, for generalization, activation functions like ReLU (Sigmoid is considered outdated) are used.

Here, the superscript represents the layer and i identifies the neuron in the layer. The function σ is an activation function. It could be sigmoid, ReLU, tanh, and other activation functions.

5.1.2 Training

Inference is rather straight forward, but how do we “train” a model? Well, that’s what this part of the note is all about. There are few terminologies that we need to know, and I think we can discuss them as we go along.

Definition 5.1.5

Forward pass, also known as **forward propagation**, the process of generating output with input data.

Here, it is worth noting that forward pass is different from inference in a sense that the process of inference only applies to a complete trained model while forward pass also applies to models being trained.

Let’s discuss the definitions and underlying concepts in individual process after capturing the big picture. Generally speaking, the following diagram captures the training process.

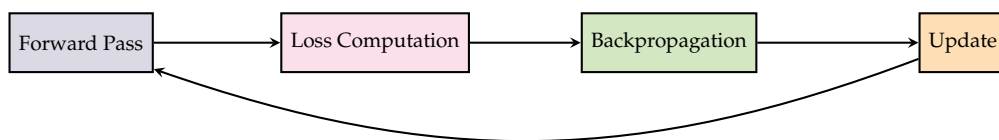


Figure 5.3: Overview of the Training Process

The first thing that we do after the forward pass is to calculate the **loss**.

Definition 5.1.6

Loss functions are algorithms that calculate the **loss**, or the performance of the prediction compared to the expected outcome.

There are many loss functions, but the two dominant functions are **mean squared error (MSE)** and **cross-entropy** (a.k.a. negative log-likelihood.) Generally, MSE is great for regression and cross-entropy excels at classification. Therefore, cross-entropy will commonly be used for LLMs for token predictions. Let's take a look at each functions.

Following the convention, let $\hat{y}_n$ and y_n be the prediction and the true value respectively. With MSE, the loss will be represented as $\frac{1}{n} \sum (\hat{y}_i - y_i)^2$. The intuition behind this formula is we always want the loss to be positive as it is comfortable to think that our goal is to reduce the loss. If the loss is negative, it will be hard to apply this sense. That's why we squared the difference. Now you might ask, why not absolute value? Indeed, there is a function Mean Absolute Error (MAE), but we like to square the values as it amplifies the loss. With cross-entropy, its a bit more complicated, but a simplified version would be $\sum (-y_i \log(\hat{y}_i))$. More on cross-entropy will be discussed soon!

We know we can calculate the loss, but how do we actually improve the model with the calculated loss?

Definition 5.1.7

An **optimizer** is an algorithm that determines the necessary changes for model weights.

The dominant optimizer type is **gradient descent** with its variants like Adam, RMSProp, and SGD. There's quite a bit of content for gradient descent for ML, so let's discuss that in a separate section of the note.

5.1.3 Gradient Descent and Learning Rate

Before we implement the ideas and concepts above with code, I wanted to discuss gradient descent and training more in-depth to actually train our model. The partial derivatives and mathematical definition of gradient descent was discussed in the first note. Here, we will discuss about gradient descent specifically for ML.

Definition 5.1.8

Gradient descent as an optimizer represents how the weights and biases of a model are updated based on the gradients of the loss function.

You would typically start from a random position to move in the opposite direction of the gradient of the loss function. Before I introduce mathematical equation, here are two famous analogies of a ball rolling down the hill and finding a way in a foggy forest. Starting with the first analogy, consider the diagram below.

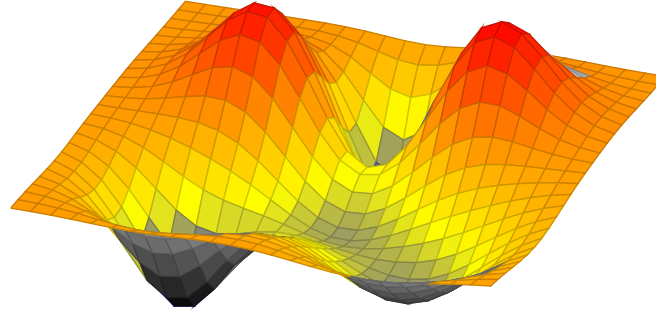


Figure 5.4: Ball Rolling Downhill Analogy

Imagine rolling a ball from the graph above. After some time, it will naturally fall into one of the pits. If we are lucky, we will find the global minimum, if not, we will fall into one of the local minima. To see how we roll a ball, consider two diagrams below.

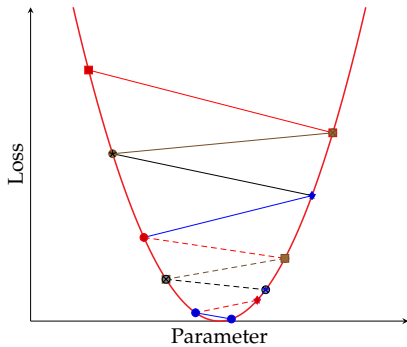


Figure 5.5: Visualization of Case I

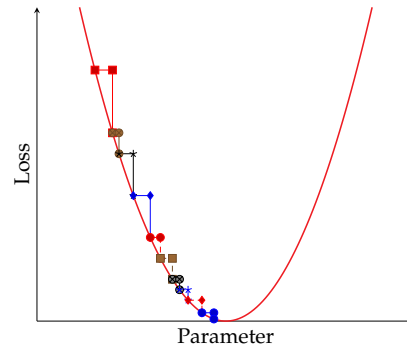


Figure 5.6: Visualization of Case II

Taking a look at Figure 5.5, imagine your random point in the red square. Because the slope of the line tangent to the point is negative, you want to move to the right, or in positive direction. This time, your learning rate is high that the slope of the line tangent to the next landing position is positive. Therefore, you want to move to the left, which is the negative direction. After continuing this process over and over again while adjusting the learning rate, you will likely find the local minimum. In Figure 5.6, our learning rate is low. Despite being low, it will still lead to the local minimum. One thing to note is that you don't want your learning rate to be low or high as you might need to train forever or overshoot. Learning rate is often denoted as η , and we normally would adjust it with the gradient, which we will see in a bit. Of course in real training, we won't be finding the slope of the tangent line as we do in the 2-dimensional plane. In higher dimension, we would find the gradient. Please refer to the first note for more information!

Another very popular analogy is foggy mountain analogy. Imagine you are in the top of the mountain, and it's getting dark. You want to get to the lowest point as fast as possible. However, you can't see anything beyond your local information because the mountain is very foggy. What would you do? We would find the gradient and multiply it with the learning rate to determine our step size. Then, with your local information, we would be able to approach the local minimum of the foggy mountain. Ideally, we should be hoping that we reach the lowest

point and villages.

Now, before we continue for building our own neural net without machine learning libraries, let's conclude the concepts by discussing different types of gradient descent and their effects.

Types of Gradient Descent

There are many types of gradient descents like Batch GD, Mini-batch GD, Stochastic GD, and Momentum-Based GD. However, the three major types are Batch Gradient Descent (BGD), Stochastic Gradient Descent (SGD), and Mini-batch Gradient Descent. On the high level, consider the following definitions.

Definition 5.1.9

With **Batch Gradient Descent (BGD)**, each update will occur based on the entire training examples, which are often averaged. In contrast, **Stochastic Gradient Descent (SGD)** updates based on a single data point. **Mini-batch Gradient Descent** is the middle ground of BGD and SGD that it updates based on few data points like 32 or 64, usually in a power of 2.

From the sense, we can notice that the biggest difference in use case would be efficiency and accuracy. BGD is generally more accurate in each update than SGD as it is based on the entire data points. However, SGD is generally more efficient as it does not have to perform massive computations for each update to incorporate all the training examples. One interesting thing to note is that because SGD is “stochastic”, or more noisy, it is sometimes better at optimizing the updates as it moves more randomly than BGD.

When visualizing, we often consider the following diagram.

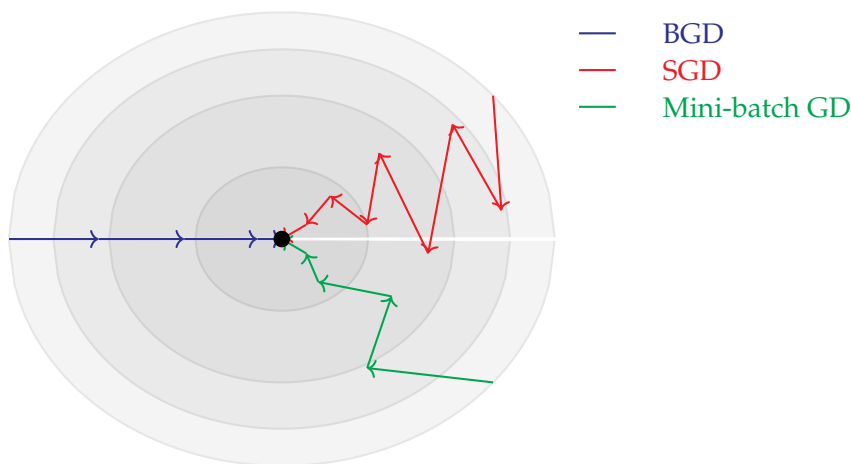


Figure 5.7: Visualization of Different Types of Gradient Descent

As you can see, BGD often heads straightly towards the minimum as the entire training examples are incorporated. Unlike BGD, SGD shows more random and noisy movement and often requires more updates to reach the local minimum. Finally, Mini-batch GD is like the

movement in between those two. It's somewhat random, but less noisy SGD than it often requires less updates.

Overall, this is what artificial neural nets are about. The pre-trained and inter-connected neurons work together to predict that output as a giant function. This giant function is composed of weights and biases, which are fine-tuned during the training process. During training, you calculate the loss and tweak the parameters to minimize the loss through gradient descent. You repeat the training process until your model learned well enough, but not too much to prevent overfitting. That's pretty much it in the high level! For our next section, let's discuss more on training and gradient descent before we actually try and build a neural network from scratch without ML libraries like PyTorch and TensorFlow.

Congratulations! We just finished the high level overview of what neural nets are and what vanilla neural nets are composed of. Before we actually try and implement the concepts with Python to build a vanilla neural net without machine learning libraries like PyTorch and TensorFlow, let's take a look at mathematical equations to write them in code.

§5.2 Conventions and Mathematical Interpretations

If you have not already read the first note on math for ML, please do so as this section of the note does not include explanations for mathematical concepts, but their interpretation here with neural networks.

Starting off, let's start with some conventions with parameters for the following neural net with two inputs, two hidden layers with four neurons respectively, and a single output.

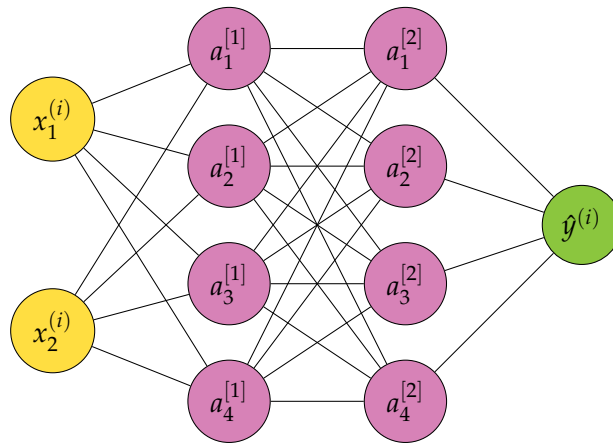


Figure 5.8: A visualization of a neural network with layers of 2, 8, 8, and 1

As you can see in the diagram above, the input of the i^{th} training example is represented with the notation $x_n^{(i)}$ for integer n as 1 through the total number of inputs. Similarly, we usually count the first hidden layer as the first layer, and the activations are represented as $a_n^{[l]}$ where l represents the layer and n again as the integer from 1 through the total number of neurons in the layer. Finally, we use $\hat{y}^{(i)}$ to represent the prediction for the i^{th} training example.

Following the convention, we could write a single neuron as the following. Note that you might be familiar with σ instead of g , but here let's use g for activation function as σ usually represents sigmoid.

Theorem 5.2.1: Formula for a Neuron

For $z_i^{[l]}$ defined as the following, the activation of a neuron can be represented as the following.

$$z_i^{[l]} = \sum_j w_{ij}^{[l]} a_j^{[l-1]} + b_i^{[l]}$$

$$a_i^{[l]} = g(z_i^{[l]}) = g\left(\sum_j w_{ij}^{[l]} a_j^{[l-1]} + b_i^{[l]}\right)$$

One reason why linear algebra is so important in machine learning is because we could represent the entire layer with simple matrix operations. Consider the following example for the second layer.

$$\begin{bmatrix} w_{11}^{[2]} & w_{12}^{[2]} & w_{13}^{[2]} & w_{14}^{[2]} \\ w_{21}^{[2]} & w_{22}^{[2]} & w_{23}^{[2]} & w_{24}^{[2]} \\ w_{31}^{[2]} & w_{32}^{[2]} & w_{33}^{[2]} & w_{34}^{[2]} \\ w_{41}^{[2]} & w_{42}^{[2]} & w_{43}^{[2]} & w_{44}^{[2]} \end{bmatrix} \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix} + \begin{bmatrix} b_1^{[2]} \\ b_2^{[2]} \\ b_3^{[2]} \\ b_4^{[2]} \end{bmatrix} = \begin{bmatrix} a_1^{[2]} \\ a_2^{[2]} \\ a_3^{[2]} \\ a_4^{[2]} \end{bmatrix}$$

Similarly, we could write different notations for updates.

Theorem 5.2.2: Adjusting Parameters with Gradient Descent

Let l_i , b_i , J , and η represent weight, bias, cost function, and learning rate respectively. The new weight and bias can be represented with the following formula.

$$w_2 = w_1 - \eta \frac{\partial J}{\partial w_1}$$

$$b_2 = b_1 - \eta \frac{\partial J}{\partial b_1}$$

As you can see above, we normally use the cost function J than the loss function L as the cost function captures the entire loss while the loss function only represent the loss per example. Now that we discussed the basic notations and mathematical formulas for the concepts that we discussed, let's take a look at math in backpropagation.

5.2.1 Deriving Equations and Backpropagation

Personally it took me a while to fully understand backprop to the state where I would be able to build a vanilla neural net from scratch. Here, I would like to share more in-depth understanding of backpropagation across numerous layers.

To be honest, it took me few days to understand it since I decided to study mainly because I didn't understand the delta rule. After reading this note, I highly recommend please try and write your own network without assistance from LLMs. I thought I understood backprop, which in reality was only the last layer. That understanding is probably all you need to build the first thing that we will create in the next section. However, I was stuck when trying to do backprop across numerous layers. To discuss this, I thought it would be helpful for us to actually rebuild or derive main equations involved in a neural network and backprop, so let's start with that. If you haven't already, please read the first note if you are unfamiliar with derivatives of multivariable functions as I won't be going over them here.

Before we derive generalized equations, let's start with a simple neural net that has two hidden layers, 2 inputs, 3 neurons in each hidden layer, and two outputs. That way, we can track the indices for matrices.

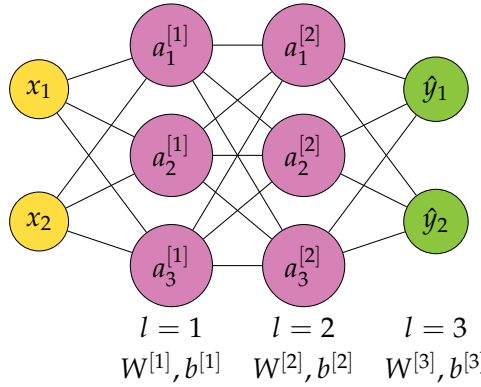


Figure 5.9: A visualization of a neural network with layers of 2, 3, 3, and 2

For simplicity, let's use MSE for our loss function, but we will be generalizing it later. Since $L_1 = \frac{1}{2}(\hat{y}_1 - y_1)^2$ and $L_2 = \frac{1}{2}(\hat{y}_2 - y_2)^2$, the following equation is obtained.

$$L = \sum_{k=1}^2 \frac{1}{2} (\hat{y}_k - y_k)^2$$

Moreover, the preactivation output of the second hidden layer can be written as following.

$$\begin{bmatrix} z_1^{[2]} \\ z_2^{[2]} \\ z_3^{[2]} \end{bmatrix} = \begin{bmatrix} w_{11}^{[2]} & w_{12}^{[2]} & w_{13}^{[2]} \\ w_{21}^{[2]} & w_{22}^{[2]} & w_{23}^{[2]} \\ w_{31}^{[2]} & w_{32}^{[2]} & w_{33}^{[2]} \end{bmatrix} \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \end{bmatrix} + \begin{bmatrix} b_1^{[2]} \\ b_2^{[2]} \\ b_3^{[2]} \end{bmatrix}$$

From the matrix multiplication above, we can write a single output as the following.

$$\begin{aligned} z_2^{[2]} &= w_{21}^{[2]} a_1^{[1]} + w_{22}^{[2]} a_2^{[1]} + w_{23}^{[2]} a_3^{[1]} + b_2^{[2]} \\ &= \sum_{j=1}^3 w_{2j}^{[2]} a_j^{[1]} + b_2^{[2]} \end{aligned}$$

Now keep in mind the indices! The 2 in w_{2j} and b_2 is the same 2 as z_2 . Generalizing for our cases, we can write the preactivation outputs as the following.

$$z_i^{[2]} = \sum_{j=1}^3 w_{ij}^{[2]} a_j^{[1]} + b_i^{[2]}$$

Nice, now that we have the forward pass for our examples, let's generalize all equations that we have now. For MSE, we could write it as the following.

$$L = \sum_k \frac{1}{2} (\hat{y}_k - y_k)^2$$

For individual preactivation output, we could write it as the following equation where n represents the number of neurons in layer $l - 1$.

$$z_i^{[l]} = \sum_{j=1}^n w_{ij}^{[l]} a_j^{[l-1]} + b_i^{[l]}$$

Now with the equations in mind, let's get into a more confusing but fun part with loss and gradient. This part is important! We are not computing the loss of individual neurons. We will be computing *gradients* instead. Loss will typically be calculated once with the final outputs.

Let's start with the last layer. Remember! This is the last layer. To make it more special, let's use l_f to denote the final layer.

$$\frac{\partial L}{\partial w_{ij}^{[l_f]}} = \frac{\partial L}{\partial a_i^{[l_f]}} \frac{\partial a_i^{[l_f]}}{\partial z_i^{[l_f]}} \frac{\partial z_i^{[l_f]}}{\partial w_{ij}^{[l_f]}}$$

In this chain rule here, we don't add other gradients because it is the last layer. The weights for the output layer does not go through other paths, but straight to the loss. That's why we don't add gradients. This would be different for different layers. Using the definition, MSE, and activation function g , the following equation is obtained.

$$\frac{\partial L}{\partial w_{ij}^{[l_f]}} = (a_i^{[l_f]} - y_i) g'(z_i^{[l_f]}) a_j^{[l_f-1]} = \delta_i^{[l_f]} a_j^{[l_f-1]}$$

Notice that $\delta_i^{[l_f]}$ will be different depending on the loss function that we use. To address that and to make our lives easier for hidden layers, we define $\delta_i^{[l]}$ as the following.

$$\delta_i^{[l]} = \frac{\partial L}{\partial z_i^{[l]}} = \frac{\partial L}{\partial a_i^{[l]}} \frac{\partial a_i^{[l]}}{\partial z_i^{[l]}}$$

Again, because the path from z_i to a_i does not require additional paths, we do not add other gradients. Now here comes the **delta rule**. Let's start deriving them together. Consider the

following equation.

$$\begin{aligned}\frac{\partial L}{\partial a_i^{[l]}} &= \sum_j \left(\frac{\partial L}{\partial z_j^{[l+1]}} \frac{\partial z_j^{[l+1]}}{\partial a_i^{[l]}} \right) \\ &= \sum_j \delta_j^{[l+1]} w_{ji}^{[l]} = \sum_j w_{ji}^{[l]} \delta_j^{[l+1]}\end{aligned}$$

Substituting back to the definition of $\delta_i^{[l]}$, the following equation is obtained by the **delta rule**.

$$\delta_i^{[l]} = g'(z_i^{[l]}) \sum_j w_{ji}^{[l]} \delta_j^{[l+1]}$$

However, writing all that for each gradient is quite tedious. Therefore, we get our general formula shown below, which will be used to build our vanilla MLP.

$$\frac{\partial L}{\partial w_{ij}^{[l]}} = \delta_i^{[l]} a_j^{[l-1]}$$

That was for the weights. Good news is that bias is more simple! Consider the following equation.

$$\frac{\partial L}{\partial b_i^{[l]}} = \frac{\partial L}{\partial a_i^{[l]}} \frac{\partial a_i^{[l]}}{\partial z_i^{[l]}} \frac{\partial z_i^{[l]}}{\partial b_i^{[l]}} = \delta_i^{[l]} \frac{\partial z_i^{[l]}}{\partial b_i^{[l]}} = \delta_i^{[l]}$$

Now with that formulas and learning rate η , we can update the weights and biases as shown in Theorem 5.2.2. There are more advanced concepts like Jacobian matrix for ease in representing, but this is pretty much it with backprop. Calculating the gradients and updating based on the loss.

§5.3 Building Vanilla Neural Network From Scratch

For our toy examples, we will create two things. The first one is technically not a neural network as it does not contain conventional neurons and structures. The second one is an actual neural net that is very simple. In this section, I will share the entire code first, then provide explanations.

5.3.1 Simple Stochastic Gradient Descent-Based Learning Engine

What we will make now is a simple stochastic gradient descent-based learning engine. This isn't technically a neural network, as you will see why in the code, but I decided to include it here as making similar examples really helped me understand backprop with code.

Our task with this learning engine is simple. We are going to predict e^x for limited range with concatenated Maclaurin Series. If you recall from your calculus courses, the following equation

holds by Taylor series.

$$e^x = \sum_{n=0}^{\infty} \frac{1}{n!} x^n = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

Here, for limited x , we will be approximating e^x with the following polynomial where b is the bias and w_i are the weights.

$$e^x \approx b + w_1x + w_2x^2 + w_3x^3 + w_4x^4 + w_5x^5 + w_6x^6 + w_7x^7 + w_8x^8$$

We will get back to this on why our x should be limited, but let's see our structure first.

We will be "training" the engine with 200 examples from -1.00 to 0.99 inclusive. Then, we will be testing it with 50 validations from 1.00 to 1.49 inclusive. This backprop machine is stochastic as it will adhere to the following structure.

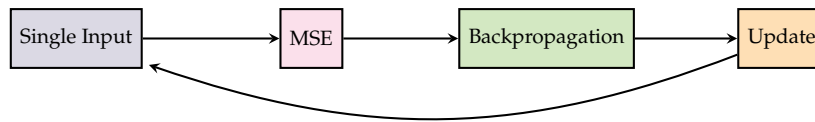


Figure 5.10: Overview of the Simple Learning Engine

As you can see, we will be using MSE for our loss function, and we do not need activation functions like sigmoid and tanh because we don't need one in our simple example. Okay enough talking and here is the entire code.

Source Code

Below is the source code for the Simple Stochastic Gradient Descent-Based Learning Engine for predicting e^x .

```

1 import math
2 import random
3 import matplotlib.pyplot as plt
4 %matplotlib inline

```

```

1 train_x = [i/100 for i in range(-100, 100)]
2 train_y = [math.exp(i/100) for i in range(-100, 100)]
3 test_x = [i/100 for i in range(100, 150)]
4 test_y = [math.exp(i/100) for i in range(100, 150)]

```

```

1 class SGDBLE:
2     def __init__(self):
3         random.seed(3141592) # So that we get the same result

```

```
4     self.ws = [random.uniform(-1, 1) for _ in range(8)]
5     self.b = random.random()
6
7     def forward(self, x):
8         self.ypred = sum(w * x**k for w, k in zip(self.ws, range(1, 9)
9             )) + self.b
10        return self.ypred
11
12    def loss(self, y, ypred):
13        self.loss_val = 0.5 * (y - ypred)**2
14        return self.loss_val
15
16    def grads(self, x, y, ypred):
17        base_der = ypred - y # i.e. dL / dypred
18        self.grads_w = [base_der * x**i for i in range(1, 9)]
19        self.grad_b = base_der
20
21    def update(self, lr):
22        for i in range(8):
23            self.ws[i] -= lr * self.grads_w[i]
24            self.b -= lr * self.grad_b
25
26    def train(self, epochs, lr):
27        losses = []
28        for epoch in range(epochs):
29            loss_total = 0
30            for i in range(len(train_x)):
31                ypred = self.forward(train_x[i])
32                loss_total += self.loss(train_y[i], ypred)
33                self.grads(train_x[i], train_y[i], ypred)
34                self.update(lr)
35            loss_avg = loss_total / len(train_x)
36            losses.append(loss_avg)
37
38            if (epoch+1) % 100 == 0:
39                print(f"Epoch: {epoch+1}/{epochs}, Loss: {loss_avg}")
40
41        return losses
42
43    def test(self):
44        loss_total = 0
45        for i in range(0, len(test_x), 2):
46            ypred1 = self.forward(test_x[i])
47            loss_total += self.loss(test_y[i], ypred1)
48
49            if i + 1 < len(test_x):
50                ypred2 = self.forward(test_x[i + 1])
```

```

50         loss_total += self.loss(test_y[i + 1], ypred2)
51
52         print(f"Test {i+1}                                Test {i+2}")
53         print(f"y:      {test_y[i]}          y:      {test_y[i+1]}"
54               )
55         print(f"ypred: {ypred1}          ypred: {ypred2}")
56         print()
57     else:
58         print(f"Test {i+1}")
59         print(f"y:      {test_y[i]}")
60         print(f"ypred: {ypred1}")
61
62     loss_total /= len(test_x)
63
64     def print_params(self):
65         coeff = [1 / math.factorial(i+1) for i in range(8)]
66         for i in range(8):
67             print(f"w{i+1}: {self.ws[i]}, Expected: {coeff[i]}")
68             print(f"b: {self.b}, Expected: 1")
69
70     model = SGDBLE()

```

```

1  def graph_loss(losses):
2      plt.plot(losses)
3      plt.xlabel("Epoch")
4      plt.ylabel("Loss")
5      plt.show()
6
7  def graph_pred():
8      xs = [i / 10 for i in range(-100, 200)]
9      true = [math.exp(x) for x in xs]
10     pred = [model.forward(x) for x in xs]
11
12     plt.plot(xs, true, label="e^x")
13     plt.plot(xs, pred, label="Prediction")
14     plt.yscale("log")
15     plt.legend()
16     plt.show()

```

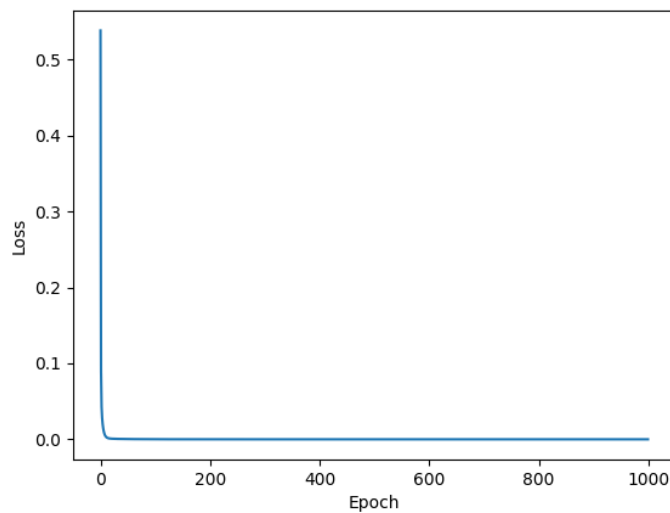
```

1  losses = model.train(1000, 0.005)
2  graph_loss(losses)
3  model.print_params()
4  graph_pred()

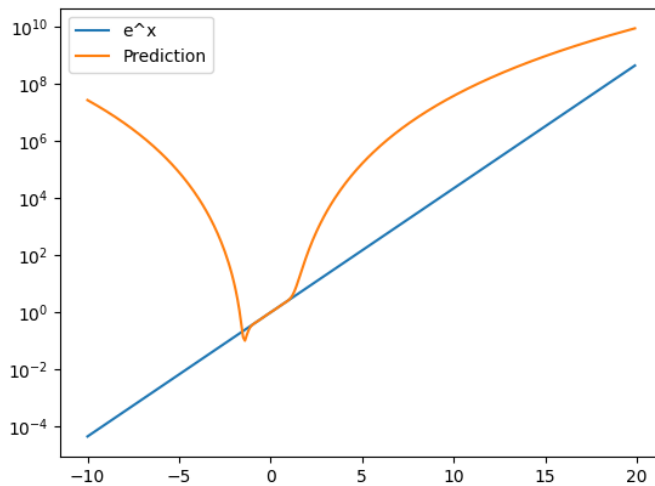
```

Epoch: 100/1000, Loss: 0.00013234985876534958

```
Epoch: 200/1000, Loss: 3.427320321762344e-05
Epoch: 300/1000, Loss: 2.4492707827970274e-05
Epoch: 400/1000, Loss: 2.2669862503550263e-05
Epoch: 500/1000, Loss: 2.172205317266906e-05
Epoch: 600/1000, Loss: 2.0984248749352728e-05
Epoch: 700/1000, Loss: 2.036538265870468e-05
Epoch: 800/1000, Loss: 1.9834658817269138e-05
Epoch: 900/1000, Loss: 1.937270927247297e-05
Epoch: 1000/1000, Loss: 1.8965386261105253e-05
```



```
w1: 0.94444454598562942, Expected: 1.0
w2: 0.49986306356857685, Expected: 0.5
w3: 0.620431907367758, Expected: 0.16666666666666666
w4: 0.21618569290559692, Expected: 0.041666666666666664
w5: -0.9321256839917001, Expected: 0.008333333333333333
w6: -0.48709437200615985, Expected: 0.0013888888888888889
w7: 0.5594822789317027, Expected: 0.0001984126984126984
w8: 0.33002989302732855, Expected: 2.48015873015873e-05
b: 0.9983077779493585, Expected: 1
```

```
1 model.test()
```

Test 1	Test 2
y: 2.718281828459045	y: 2.7456010150169163
ypred: 2.7495260176087557	ypred: 2.7867851769675642
Test 3	Test 4
y: 2.7731947639642978	y: 2.801065834699079
ypred: 2.8257757734623574	ypred: 2.8666322590606814
Test 5	Test 6
y: 2.82921701435156	y: 2.857651118063164
ypred: 2.909497162739583	ypred: 2.954521430346852
Test 7	Test 8
y: 2.8863709892679585	y: 2.915379499976997
ypred: 3.001864774074873	ypred: 3.0516960317129964
Test 9	Test 10
y: 2.944679551065524	y: 2.9742740725630656
ypred: 3.1041935358457007	ypred: 3.159545493165112
Test 11	Test 12
y: 3.0041660239464334	y: 3.034358394435676
ypred: 3.2179503740678106	ypred: 3.2796173127071624
Test 13	Test 14
y: 3.0648542032930024	y: 3.095656500124711
ypred: 3.3447665176737535	ypred: 3.4136296934778425

Test 15 y: 3.1267683651861553 ypred: 3.4864504730090555	Test 16 y: 3.158192909689767 ypred: 3.5634848611499024
Test 17 y: 3.1899332761161845 ypred: 3.6450016897210107	Test 18 y: 3.2219926385284996 ypred: 3.7312830839373126
Test 19 y: 3.2543742028896707 ypred: 3.8226249405557278	Test 20 y: 3.287081207383118 ypred: 3.9193374178962683
Test 21 y: 3.3201169227365472 ypred: 4.02174543791974	Test 22 y: 3.3534846525490236 ypred: 4.130189200546641
Test 23 y: 3.3871877336213347 ypred: 4.245024710403111	Test 24 y: 3.4212295362896734 ypred: 4.366624316181152
Test 25 y: 3.4556134647626755 ypred: 4.495377262801668	Test 26 y: 3.4903429574618414 ypred: 4.631690256570204
Test 27 y: 3.5254214873653824 ypred: 4.775988043516579	Test 28 y: 3.5608525623555205 ypred: 4.928714001110956
Test 29 y: 3.5966397255692817 ypred: 5.090330743550226	Test 30 y: 3.6327865557528094 ypred: 5.261320740809877
Test 31 y: 3.6692966676192444 ypred: 5.442186951657918	Test 32 y: 3.706173712210199 ypred: 5.633453470828671
Test 33 y: 3.7434213772608627 ypred: 5.8356661905556715	Test 34 y: 3.781043387568781 ypred: 6.049393476664138
Test 35 y: 3.819043505366336 ypred: 6.275226859424931	Test 36 y: 3.8574255306969745 ypred: 6.513781739373128
Test 37 y: 3.896193301795215 ypred: 6.765698108295758	Test 38 y: 3.9353506954704733 ypred: 7.0316412855945165

Test 39	Test 40
y: 3.9749016274947477	y: 4.014850052994201
ypred: 7.312302670230667	ypred: 7.6084005084606225
Test 41	Test 42
y: 4.0551999668446745	y: 4.095955404071176
ypred: 7.9206806775719905	ypred: 8.249917485831345
Test 43	Test 44
y: 4.137120440251392	y: 4.178699191923246
ypred: 8.596914488856152	ypred: 8.962505322624688
Test 45	Test 46
y: 4.220695816996552	y: 4.263114515168817
ypred: 9.347554553339133	ypred: 9.752958544358306
Test 47	Test 48
y: 4.305959528345206	y: 4.349235141062741
ypred: 10.179646340417854	ypred: 10.628580569357089
Test 49	Test 50
y: 4.392945680918757	y: 4.437095519003664
ypred: 11.100758361572902	ypred: 11.597212287422602

Detailed Explanation

Now let's take a look at individual cells.

```
1 import math
2 import random
3 import matplotlib.pyplot as plt
4 %matplotlib inline
```

Here, I tried to keep it as minimal as possible. Instead of NumPy, I decided to use the `math` module for all math stuffs like `math.exp()`. We need `random` as it would make our lives easier than to manually set the eight weights and the bias. Matplotlib of course since we want to see how it learns for limited range of input.

```
1 train_x = [i/100 for i in range(-100, 100)]
2 train_y = [math.exp(i/100) for i in range(-100, 100)]
3 test_x = [i/100 for i in range(100, 150)]
4 test_y = [math.exp(i/100) for i in range(100, 150)]
```

Setting up the dataset for our engine is also quite simple. Instead of writing every single number,

I just decided to loop through them and store them in an array, and that is `train_x`. The values for `train_y` is also quite straightforward as it is e^x for corresponding x . The same goes for the test set. I tried to keep it 80% and 20% for training and validation, but that is not necessary and it varies by models. However, I decided to use that here because that's pretty much what most people do when they first train. Now let's get into the fun stuffs.

```
1 class SGDBLE:
2     def __init__(self):
3         random.seed(3141592) # So that we get the same result
4         self.ws = [random.uniform(-1, 1) for _ in range(8)]
5         self.b = random.random()
```

Now I decided to store the core functions in the class SGDBLE as we will be using `self` to store attributes and objects. Here we set the specific seed so that you will also get the same result when you run the code in somewhere. Next we set weights and biases and stored them in `self` with our set random values.

```
1 def forward(self, x):
2     self.ypred = sum(w * x**k for w, k in zip(self.ws, range(1, 9))) +
3         self.b
4     return self.ypred
5
6 def loss(self, y, ypred):
7     self.loss_val = 0.5 * (y - ypred)**2
8     return self.loss_val
```

This is where we set some core functions. Forward is forward pass and it is somewhat straight forward as that is how we defined it. For the loss function, we are using MSE, i.e. $\frac{1}{2}(y - \hat{y})^2$.

```
1 def grads(self, x, y, ypred):
2     base_der = ypred - y # i.e. dL / dypred
3     self.grads_w = [base_der * x**i for i in range(1, 9)]
4     self.grad_b = base_der
5
6 def update(self, lr):
7     for i in range(8):
8         self.ws[i] -= lr * self.grads_w[i]
9     self.b -= lr * self.grad_b
```

This is the core part of our engine. Let's briefly look at the math and write the loop out for some

to see what's actually going on. Consider the following equation by the chain rule.

$$\frac{dL}{dw_i} = \frac{dL}{dy} \cdot \frac{dy}{dw_i}$$

Notice that $\frac{dL}{dy} = \frac{d}{dy}(\frac{1}{2}(y - \hat{y})^2) = y - \hat{y}$. This is `base_der`. Continuing with the chain rule, we can write the weights and the bias as the following.

$$\begin{aligned}\frac{dL}{dw_1} &= \frac{dL}{dy} \cdot \frac{dy}{dw_1} = \text{base_der} \cdot x \\ \frac{dL}{dw_2} &= \frac{dL}{dy} \cdot \frac{dy}{dw_2} = \text{base_der} \cdot x^2 \\ \frac{dL}{db} &= \frac{dL}{dy} \cdot \frac{dy}{db} = \text{base_der} \cdot 1\end{aligned}$$

Now the update function. Recall $w_2 = w_1 - \eta \frac{dL}{dw_1}$ and $b_2 = b_1 - \eta \frac{dL}{db_1}$. That is exactly what we did, but for weights we have to loop through each element.

```

1  def train(self, epochs, lr):
2      losses = []
3      for epoch in range(epochs):
4          loss_total = 0
5          for i in range(len(train_x)):
6              ypred = self.forward(train_x[i])
7              loss_total += self.loss(train_y[i], ypred)
8              self.grads(train_x[i], train_y[i], ypred)
9              self.update(lr)
10         loss_avg = loss_total / len(train_x)
11         losses.append(loss_avg)
12
13         if (epoch+1) % 100 == 0:
14             print(f"Epoch: {epoch+1}/{epochs}, Loss: {loss_avg}")
15
16     return losses

```

This is the training loop. Of course we don't want to stop at a single epoch especially in stochastic setting. You can ignore the `losses` in the beginning as we only use them to graph. For each epoch, we want to set the total loss to 0 as they will accumulate. Then, for all x values in our training set, we will predict, calculate the loss, backprop, an update as shown. Note that we have to set `epoch` and `lr`, which is the learning rate. Now we will take the average of the total loss and print it every 100 epoch to see how well our engine is learning.

```

1  def test(self):
2      loss_total = 0
3      for i in range(0, len(test_x), 2):

```

```

4         ypred1 = self.forward(test_x[i])
5         loss_total += self.loss(test_y[i], ypred1)
6
7         if i + 1 < len(test_x):
8             ypred2 = self.forward(test_x[i + 1])
9             loss_total += self.loss(test_y[i + 1], ypred2)
10
11             print(f"Test {i+1}                      Test {i+2}")
12             print(f"y:      {test_y[i]}          y:      {test_y[i+1]}")
13             print(f"ypred: {ypred1}          ypred: {ypred2}")
14             print()
15         else:
16             print(f"Test {i+1}")
17             print(f"y:      {test_y[i]}")
18             print(f"ypred: {ypred1}")
19
20     loss_total /= len(test_x)
21
22     def print_params(self):
23         coeff = [1 / math.factorial(i+1) for i in range(8)]
24         for i in range(8):
25             print(f"w{i+1}: {self.ws[i]}, Expected: {coeff[i]}")
26             print(f"b: {self.b}, Expected: 1")
27
28     model = SGDBLE()

```

The test loop is also straight forward. It predicts, calculates the loss, and print them as shown. Also, because we know the coefficients for the Maclaurin Series, I decided to see if the parameters match with the actual coefficients. Finally, the last line is to be able use the model.

```

1     def graph_loss(losses):
2         plt.plot(losses)
3         plt.xlabel("Epoch")
4         plt.ylabel("Loss")
5         plt.show()
6
7     def graph_pred():
8         xs = [i / 10 for i in range(-100, 200)]
9         true = [math.exp(x) for x in xs]
10        pred = [model.forward(x) for x in xs]
11
12        plt.plot(xs, true, label="e^x")
13        plt.plot(xs, pred, label="Prediction")
14        plt.yscale("log")
15        plt.legend()
16        plt.show()

```

This part is plotting with Matplotlib. Please refer to the second note if you are confused! One thing though `yscale` is necessary as exponential function will get too large to show.

```
1 losses = model.train(1000, 0.005)
2 graph_loss(losses)
3 model.print_params()
4 graph_pred()
```

This part is the actual training part. We set the epoch to 1000 and learning rate to 0.005. This learning rate might be too small for the start, but I realized that this gives slightly better results than 0.001 in this setting. Then we graph and show loss, parameters, and predictions.

```
1 model.test()
```

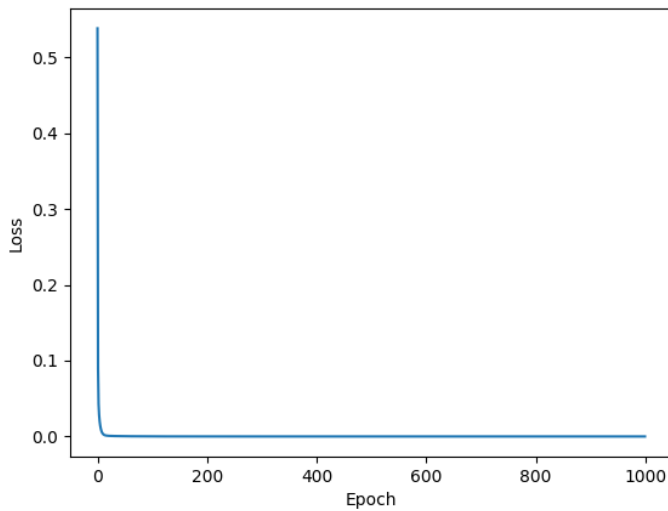
This part is for testing. Now that we have seen the entire code and explanation for it, let's now analyze our results.

Analyzing the Results

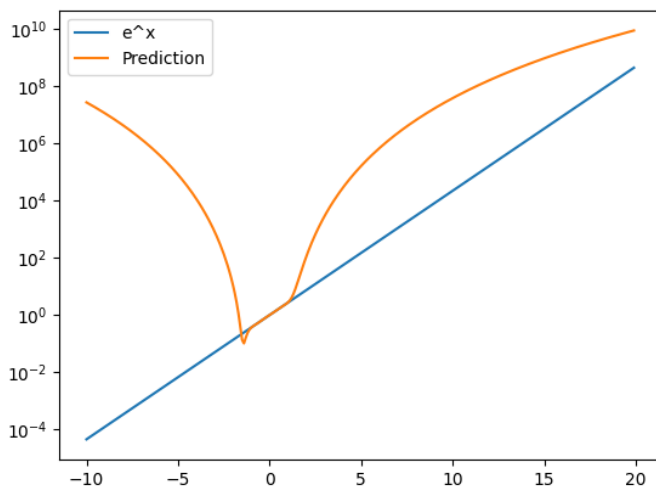
First, let's take a look at the first result.

```
1 losses = model.train(1000, 0.005)
2 graph_loss(losses)
3 model.print_params()
4 graph_pred()

Epoch: 100/1000, Loss: 0.00013234985876534958
Epoch: 200/1000, Loss: 3.427320321762344e-05
Epoch: 300/1000, Loss: 2.4492707827970274e-05
Epoch: 400/1000, Loss: 2.2669862503550263e-05
Epoch: 500/1000, Loss: 2.172205317266906e-05
Epoch: 600/1000, Loss: 2.0984248749352728e-05
Epoch: 700/1000, Loss: 2.036538265870468e-05
Epoch: 800/1000, Loss: 1.9834658817269138e-05
Epoch: 900/1000, Loss: 1.937270927247297e-05
Epoch: 1000/1000, Loss: 1.8965386261105253e-05
```



```
w1: 0.9444454598562942, Expected: 1.0
w2: 0.49986306356857685, Expected: 0.5
w3: 0.620431907367758, Expected: 0.16666666666666666
w4: 0.21618569290559692, Expected: 0.041666666666666664
w5: -0.9321256839917001, Expected: 0.008333333333333333
w6: -0.48709437200615985, Expected: 0.0013888888888888889
w7: 0.5594822789317027, Expected: 0.0001984126984126984
w8: 0.33002989302732855, Expected: 2.48015873015873e-05
b: 0.9983077779493585, Expected: 1
```



This is what we got after the first training. The loss looks pretty good actually. As you can see for each 100th epoch, the loss is decreasing well. From the graph, you can see the sudden drop in losses; however, we can't necessarily say that this is a grokking phenomena as it might just be model understanding the pattern from underfitting.

Looking at the weights, some are good and some are bad. We will get back to this later, but

this is also one of the reason why we cannot scale this engine. Similarly, we can see that our polynomial fits very well for the small range, but not well as the input increases.

```
1 model.test()
```

Test 1	Test 2
y: 2.718281828459045	y: 2.7456010150169163
ypred: 2.7495260176087557	ypred: 2.7867851769675642
Test 3	Test 4
y: 2.7731947639642978	y: 2.801065834699079
ypred: 2.8257757734623574	ypred: 2.8666322590606814
Test 5	Test 6
y: 2.82921701435156	y: 2.857651118063164
ypred: 2.909497162739583	ypred: 2.954521430346852
...	
Test 45	Test 46
y: 4.220695816996552	y: 4.263114515168817
ypred: 9.347554553339133	ypred: 9.752958544358306
Test 47	Test 48
y: 4.305959528345206	y: 4.349235141062741
ypred: 10.179646340417854	ypred: 10.628580569357089
Test 49	Test 50
y: 4.392945680918757	y: 4.437095519003664
ypred: 11.100758361572902	ypred: 11.597212287422602

For the training set, I only listed some of them as you can refer to the previous pages. Notice that for first few tests, the predictions are not bad. However, for Test 5, the engine does not seem be effective and seems lost from Test 10. We could actually run our training loop multiple times to see what happens, and I will leave this to the readers. However, the engine does not seem to learn well.

Here is the question. Is the engine overfitting right now? If you said no, well... that is correct! Technically, we can see it as overfitting, but what I think is more accurate is to say that this is the limitation of the scalability of this engine and our method. When predicting, we only used nine terms. From the parameters, some were very close to the expectation and others were very bad. This is because we are using a polynomial with finite term to predict, whereas the real Maclaurin series uses infinite sum. Therefore, it is technically an overfitting to certain point in a way that our engine tries to fit our polynomial in limited range as shown in the graph, but this too could improve if we have the computational ability to use infinite sum. Therefore, it may be more accurate to say that it is the limitation of the engine and scalability that causes high loss in testing.

We just completed our first model! Although it is very limited in terms of scalability and usability, I hope this example was very helpful in using backpropagation in code. For our next example, we will build an actual simple neural net to predict a nonlinear function.

5.3.2 Vanilla Neural Network

Now that we have created a simple backprop engine, let's increase the number of hidden layers to predict x_1x_2 for inputs x_1 and x_2 . I thought it would be great to introduce nonlinearity to our problem, and it works! Without further ado, let's get started by taking a look at the source code.

Source Code

Below is the entire source code with the results.

```
1 import math
2 import random
3 import matplotlib.pyplot as plt
4 %matplotlib inline
```

```
1 # Predicting  $x_1 * x_2$ 
2 def tanh(x):
3     return (math.exp(x) - math.exp(-x)) / (math.exp(x) + math.exp(-x))
4
5 def tanh_der(x):
6     return 1 - tanh(x)**2
7
8 def loss(y, ypred):
9     return 0.5 * (y - ypred)**2
10
11
12 # Data
13 data_x = [[i/10, j/10] for i in range(-10, 11) for j in range(-10, 11)]
14 data_y = [(x1 * x2) for x1, x2 in data_x]
15
16 data = list(zip(data_x, data_y))
17 random.seed(3141592)
18 random.shuffle(data)
19
20 split = int(0.8 * len(data))
21 train_data = data[:split]
22 test_data = data[split:]
23
24 train_x = [x for x, y in train_data]
```

```
25 train_y = [y for x, y in train_data]
26 test_x = [x for x, y in test_data]
27 test_y = [y for x, y in test_data]
```

```
1 class MLP:
2     def __init__(self, layers):
3         self.ws = []
4         self.bs = []
5
6         random.seed(3141592)
7         for i in range(len(layers) - 1):
8             layer_in = layers[i]
9             layer_out = layers[i + 1]
10            layer_w = [[random.uniform(-1, 1) for _ in range(layer_in)
11                        for _ in range(layer_out)]]
12            layer_b = [random.uniform(-1, 1) for _ in range(layer_out)
13                      ]
14
15            self.ws.append(layer_w)
16            self.bs.append(layer_b)
17
18        def forward(self, x):
19            self.act = [x]
20            self.zs = []
21
22            # Layers
23            for i in range(len(self.ws)):
24                prev = self.act[-1]
25                pre_layer = []
26                act_layer = []
27
28                # Neurons
29                for j in range(len(self.ws[i])):
30                    pre = self.bs[i][j]
31                    for k in range(len(self.ws[i][j])):
32                        pre += self.ws[i][j][k] * prev[k]
33
34                    pre_layer.append(pre)
35
36                if i != len(self.ws) - 1:
37                    act_layer.append(tanh(pre))
38                else:
39                    act_layer.append(pre)
40
41            self.zs.append(pre_layer)
```

```

41         self.act.append(act_layer)
42
43     return self.act[-1][0]
44
45 def backprop(self, y):
46     self.grad_w = [[[0.0 for _ in range(len(self.ws[i][j]))]
47                     for j in range(len(self.ws[i]))]
48                    for i in range(len(self.ws))]
49     self.grad_b = [[0.0 for _ in range(len(self.bs[i]))]
50                   for i in range(len(self.bs))]
51     ypred = self.act[-1][0]
52     delta = [0] * len(self.ws)
53     delta[-1] = [ypred - y]
54
55     # range(start, stop, step)
56     for k in range(len(self.ws) - 2, -1, -1):
57         delta[k] = []
58         for i in range(len(self.ws[k])):
59             dlda = 0.0
60             for j in range(len(self.ws[k + 1])):
61                 dlda += self.ws[k + 1][j][i] * delta[k + 1][j]
62             delta[k].append(dlda * tanh_der(self.zs[k][i]))
63
64     for i in range(len(self.ws)):
65         prev = self.act[i]
66         for j in range(len(self.ws[i])):
67             self.grad_b[i][j] = delta[i][j]
68             for k in range(len(prev)):
69                 self.grad_w[i][j][k] = delta[i][j] * prev[k]
70
71 def update(self, lr):
72     for i in range(len(self.ws)):
73         for j in range(len(self.ws[i])):
74             for k in range(len(self.ws[i][j])):
75                 self.ws[i][j][k] -= lr * self.grad_w[i][j][k]
76                 self.bs[i][j] -= lr * self.grad_b[i][j]
77
78 def train(self, train_x, train_y, epochs, lr):
79     losses = []
80     for epoch in range(epochs):
81         total_loss = 0
82
83         for x, y in zip(train_x, train_y):
84             ypred = self.forward(x)
85             total_loss += loss(y, ypred)
86             self.backprop(y)
87             self.update(lr)

```

```

88         average_loss = total_loss / len(train_x)
89         losses.append(average_loss)
90
91     if (epoch + 1) % 100 == 0:
92         print(f"Epoch: {epoch+1}/{epochs}, Loss: {
93             average_loss}")
94
95     return losses
96
97
98     def show_params(self):
99         for i, (w, b) in enumerate(zip(self.ws, self.bs)):
100             layer = "OUTPUT" if i == len(self.ws) - 1 else f"LAYER {i
101                 +1}"
102             print(layer)
103             for j, neuron_w in enumerate(w):
104                 weights_str = ", ".join(f"{x:.4f}" for x in neuron_w)
105                 print(f"    Neuron {j+1}:")
106                 print(f"        Weights: [{weights_str}]")
107                 print(f"        Bias = {b[j]:.4f}")
108             print()
109 mlp = MLP([2, 8, 8, 1])

```

```

1 losses = mlp.train(train_x, train_y, epochs=2000, lr=0.01)
2 print()
3
4 for x, y in zip(train_x[:10], train_y[:10]):
5     print(x, y)
6     print(f"    ypred: {mlp.forward(x)}")
7
8 plt.plot(losses)
9 plt.xlabel("epoch")
10 plt.ylabel("loss")
11 plt.show()
12
13 #mlp.show_params()

```

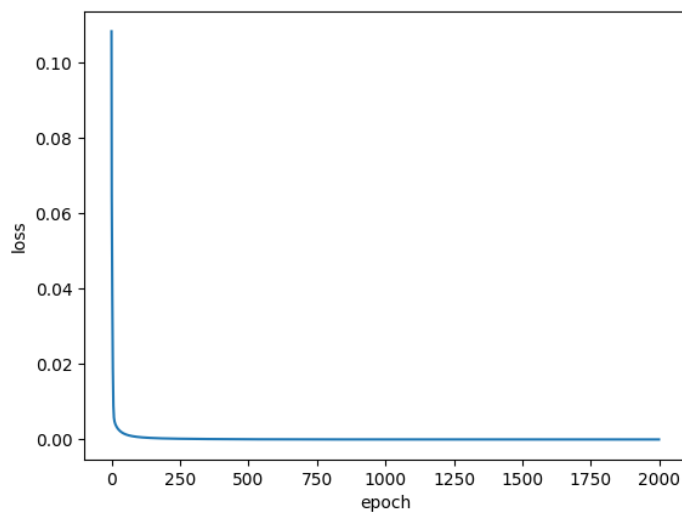
```

Epoch: 100/2000, Loss: 0.0006795237745380497
Epoch: 200/2000, Loss: 0.0002780199905385479
Epoch: 300/2000, Loss: 0.00015780133554953304
Epoch: 400/2000, Loss: 9.996884725589866e-05
Epoch: 500/2000, Loss: 6.544552166502369e-05
Epoch: 600/2000, Loss: 4.373246439583505e-05
Epoch: 700/2000, Loss: 3.032175895325618e-05

```

```
Epoch: 800/2000, Loss: 2.2295750007135315e-05
Epoch: 900/2000, Loss: 1.7559747007674843e-05
Epoch: 1000/2000, Loss: 1.4723606896641237e-05
Epoch: 1100/2000, Loss: 1.2951440585334302e-05
Epoch: 1200/2000, Loss: 1.1773900112740833e-05
Epoch: 1300/2000, Loss: 1.0936643742787753e-05
Epoch: 1400/2000, Loss: 1.0302935457567813e-05
Epoch: 1500/2000, Loss: 9.79811628836463e-06
Epoch: 1600/2000, Loss: 9.379957580934361e-06
Epoch: 1700/2000, Loss: 9.023360655331247e-06
Epoch: 1800/2000, Loss: 8.712547584218995e-06
Epoch: 1900/2000, Loss: 8.437042172730164e-06
Epoch: 2000/2000, Loss: 8.189547717119376e-06
```

```
[1.0, -1.0] -1.0
      ypred: -0.9914314084230468
[0.8, -1.0] -0.8
      ypred: -0.8026975789984329
[0.8, -0.9] -0.7200000000000001
      ypred: -0.7229659139930072
[0.1, -0.8] -0.08000000000000002
      ypred: -0.07901899113208088
[0.3, -0.3] -0.09
      ypred: -0.08931568886644314
[-0.7, -0.1] 0.06999999999999999
      ypred: 0.07003157242570901
[-0.6, -0.5] 0.3
      ypred: 0.29863205953269367
[0.7, 0.8] 0.5599999999999999
      ypred: 0.5613061391881997
[-0.5, -0.4] 0.2
      ypred: 0.19629911113434118
[0.9, 0.8] 0.7200000000000001
      ypred: 0.7218073084017129
```



```

1 test_loss = 0
2 for x, y in zip(test_x, test_y):
3     ypred = mlp.forward(x)
4     test_loss += loss(y, ypred)
5
6 for x, y in zip(test_x[:10], test_y[:10]):
7     ypred = mlp.forward(x)
8     print(x, y)
9     print(f"    ypred: {ypred}")
10
11 print("Test Loss:", test_loss / len(test_x))

```

```

[-0.9, -0.3] 0.27
    ypred: 0.27224246623233583
[0.9, 1.0] 0.9
    ypred: 0.889162612694173
[0.1, 0.5] 0.05
    ypred: 0.050824340568330406
[1.0, 1.0] 1.0
    ypred: 0.9737260427905209
[0.6, -0.8] -0.48
    ypred: -0.4811987339447982
[0.6, -0.3] -0.18
    ypred: -0.17587974783858806
[0.6, 0.4] 0.24
    ypred: 0.23577905969192914
[0.5, -0.8] -0.4
    ypred: -0.4007379739663608
[0.5, 0.2] 0.1
    ypred: 0.0970149401772119

```

```
[0.9, -0.2] -0.18000000000000002
ypred: -0.18071247280108016
Test Loss: 1.1432408362631248e-05
```

Detailed Explanation

Now let's take a look at each block of the code to see their role in our vanilla neural net.

```
1 import math
2 import random
3 import matplotlib.pyplot as plt
4 %matplotlib inline
```

Here, I imported essential modules and libraries to do math, get a randomized value, and plot the loss.

```
1 # Predicting x1 * x2
2 def tanh(x):
3     return (math.exp(x) - math.exp(-x)) / (math.exp(x) + math.exp(-x))
4
5 def tanh_der(x):
6     return 1 - tanh(x)**2
7
8 def loss(y, ypred):
9     return 0.5 * (y - ypred)**2
```

For this problem, I decided to use tanh for our activation function as sigmoid squishes the values to 0 and 1, which is not ideal for regression problem like ours and ReLU was not effective when I tried. Since $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ and $\tanh'(x) = 1 - \tanh^2(x)$, we did the same. For our loss function, I decided to use MSE for simplicity again.

```
1 # Data
2 data_x = [[i/10, j/10] for i in range(-10, 11) for j in range(-10, 11)]
3 data_y = [(x1 * x2) for x1, x2 in data_x]
4
5 data = list(zip(data_x, data_y))
6 random.seed(3141592)
7 random.shuffle(data)
8
9 split = int(0.8 * len(data))
10 train_data = data[:split]
11 test_data = data[split:]
```



```

12
13 train_x = [x for x, y in train_data]
14 train_y = [y for x, y in train_data]
15 test_x = [x for x, y in test_data]
16 test_y = [y for x, y in test_data]

```

This part is for making our datasets. I decided to divide i and j by 10 because if I don't, math overflow error will be resulted due to tanh function.

This time, I decided to shuffle data in a way that we will get the same result when you try running this, and split the data into 80% for training and 20% for validating.

```

1 class MLP:
2     def __init__(self, layers):
3         self.ws = []
4         self.bs = []
5
6         random.seed(3141592)
7         for i in range(len(layers) - 1):
8             layer_in = layers[i]
9             layer_out = layers[i + 1]
10            layer_w = [[random.uniform(-1, 1) for _ in range(layer_in
11                                                                )]
12                       for _ in range(layer_out)]
13            layer_b = [random.uniform(-1, 1) for _ in range(layer_out
14                                                                )]
15
16            self.ws.append(layer_w)
17            self.bs.append(layer_b)

```

Again for our example, I decided to store our core functions in a class since we want to reuse and store our attributes. When this class is first called, we will create an empty array and for `ws` and `bs` and store them in `self`. Then, for the randomized value, we will loop through each layers to store a randomized value between -1 and 1 for weights and biases. Notice that unlike the bias which only loop through the neurons in the output layer, we have to loop through the previous layer too for weights since we have different weights for each previous neuron for each output neuron. We will then append, or store, our layer to `ws` and `bs`.

```

1 def forward(self, x):
2     self.act = [x]
3     self.zs = []
4
5     # Layers
6     for i in range(len(self.ws)):

```

```

7     prev = self.act[-1]
8     pre_layer = []
9     act_layer = []
10
11     # Neurons
12     for j in range(len(self.ws[i])):
13         pre = self.bs[i][j]
14         for k in range(len(self.ws[i][j])):
15             pre += self.ws[i][j][k] * prev[k]
16
17         pre_layer.append(pre)
18
19         if i != len(self.ws) - 1:
20             act_layer.append(tanh(pre))
21         else:
22             act_layer.append(pre)
23
24     self.zs.append(pre_layer)
25     self.act.append(act_layer)
26
27     return self.act[-1][0]

```

This is the forward function. Here, I decided to create both `self.act` and `self.zs` since we need both of them for backprop function. As the name suggests, `self.act` is the value after tanh is applied to `self.zs`. Moreover, I stored the input in `self.act` in the beginning to start using them in the first hidden layer.

Continuing, we create arrays for each layer for preactivation and activation. Then, we will continue by looping through the neurons within each layer. Our preactivation will start from the bias, and by following the definition, we will add the products of weights and the previous activation. For the if-else condition, I decided to do this because we don't want tanh for our final output layer. Then, we will store the layers in `self.act` and `self.zs` for latter use.

```

1  def backprop(self, y):
2      self.grad_w = [[0.0 for _ in range(len(self.ws[i][j]))]
3                      for j in range(len(self.ws[i]))
4                      for i in range(len(self.ws))]
5      self.grad_b = [[0.0 for _ in range(len(self.bs[i]))]
6                      for i in range(len(self.bs))]
7      ypred = self.act[-1][0]
8      delta = [0] * len(self.ws)
9      delta[-1] = [ypred - y]
10
11     # range(start, stop, step)
12     for k in range(len(self.ws) - 2, -1, -1):

```

```

13     delta[k] = []
14     for i in range(len(self.ws[k])):
15         dlda = 0.0
16         for j in range(len(self.ws[k + 1])):
17             dlda += self.ws[k + 1][j][i] * delta[k + 1][j]
18             delta[k].append(dlda * tanh_der(self.zs[k][i]))
19
20     for i in range(len(self.ws)):
21         prev = self.act[i]
22         for j in range(len(self.ws[i])):
23             self.grad_b[i][j] = delta[i][j]
24             for k in range(len(prev)):
25                 self.grad_w[i][j][k] = delta[i][j] * prev[k]

```

Ah the backprop. I think this is the hardest part from our code. Assuming that you have read the previous section, let's interpret the code with the corresponding math equations.

Recall although we do not calculate the loss per individual parameter, we do have to calculate the gradients. To start, I decided to put the initial values as 0. To store them, we can do the same just as we randomized the initial weights and biases because ultimately, each gradient will have a matching parameter. Continuing, y_{pred} is the prediction, and that is the first element from the last activation layer. This will vary when we have numerous outputs, but since this is a simple regression problem and we only have one output, this is good. δ here is the delta term for our hidden layers. Let's start by setting them as 0 at first and $y_{pred} - y$ for the last layer.

Here comes the fun part. In Python, the `range` function can be used as `range(start, stop, step)`. Here, we start from the last hidden layer and we subtract 2 due to indices. Then, we will stop when we reach -1 by going backward by 1. We will loop through them for our delta terms. Recall the formula that we derived below.

$$\delta_i^{[l]} = \frac{\partial L}{\partial a_i^{[l]}} \frac{\partial a_i^{[l]}}{\partial z_i^{[l]}} = g'(z_i^{[l]}) \sum_j w_{ji}^{[l]} \delta_j^{[l+1]}$$

As the formula suggests, we will add the weights and the corresponding following delta term with the for loop for $\frac{\partial L}{\partial a_i^{[l]}}$. Then, we will multiply it by the derivative of the activation function, which is $\tanh$ for our example. Finally, we can store our delta terms that are calculated. Next, we can now use them to calculate the gradients.

$$\frac{\partial L}{\partial w_{ij}^{[l]}} = \delta_i^{[l]} a_j^{[l-1]}$$

$$\frac{\partial L}{\partial b_i^{[l]}} = \delta_i^{[l]} \frac{\partial z_i^{[l]}}{\partial b_i^{[l]}} = \delta_i^{[l]}$$

As the formula suggests, for each parameters, will calculate the gradients again with for the

loop. This is it for the backprop!

```

1  def update(self, lr):
2      for i in range(len(self.ws)):
3          for j in range(len(self.ws[i])):
4              for k in range(len(self.ws[i][j])):
5                  self.ws[i][j][k] -= lr * self.grad_w[i][j][k]
6                  self.bs[i][j] -= lr * self.grad_b[i][j]
7
8  def train(self, train_x, train_y, epochs, lr):
9      losses = []
10     for epoch in range(epochs):
11         total_loss = 0
12
13         for x, y in zip(train_x, train_y):
14             ypred = self.forward(x)
15             total_loss += loss(y, ypred)
16             self.backprop(y)
17             self.update(lr)
18
19         average_loss = total_loss / len(train_x)
20         losses.append(average_loss)
21
22         if (epoch + 1) % 100 == 0:
23             print(f"Epoch: {epoch+1}/{epochs}, Loss: {average_loss}")
24
25     return losses

```

Now we need to update. This part is similar with our previous example, but for each parameter, we will update it by subtracting the product of the corresponding gradient and the learning rate. The training part is again very similar to our previous example. We will begin our loss with 0, then we will predict, accumulate the loss, backprop, update, and print every hundred epoch.

```

1  def show_params(self):
2      for i, (w, b) in enumerate(zip(self.ws, self.bs)):
3          layer = "OUTPUT" if i == len(self.ws) - 1 else f"LAYER {i+1}"
4          print(layer)
5          for j, neuron_w in enumerate(w):
6              weights_str = ", ".join(f"{x:.4f}" for x in neuron_w)
7              print(f"    Neuron {j+1}:")
8              print(f"        Weights: [{weights_str}]")
9              print(f"        Bias = {b[j]:.4f}")
10     print()
11

```

```
12 mlp = MLP([2, 8, 8, 1])
```

This part is for printing the parameters, and as you can see, it is pretty straight forward. I ended up not using it because I don't think it is that effective at this point, but why not just have it. Now we create an instance for the MLP class with two hidden layers with eight neurons each.

```
1 losses = mlp.train(train_x, train_y, epochs=2000, lr=0.01)
2 print()
3
4 for x, y in zip(train_x[:10], train_y[:10]):
5     print(x, y)
6     print(f"      ypred: {mlp.forward(x)}")
7
8 plt.plot(losses)
9 plt.xlabel("epoch")
10 plt.ylabel("loss")
11 plt.show()
12
13 #mlp.show_params()
```

We will train with the learning rate of 0.01 and 2000 epochs. We then print the first ten predictions and plot the loss.

```
1 test_loss = 0
2 for x, y in zip(test_x, test_y):
3     ypred = mlp.forward(x)
4     test_loss += loss(y, ypred)
5
6 for x, y in zip(test_x[:10], test_y[:10]):
7     ypred = mlp.forward(x)
8     print(x, y)
9     print(f"      ypred: {ypred}")
10
11 print("Test Loss:", test_loss / len(test_x))
```

This part is for printing the loss and prediction for testing. That was it for the explanation! Now let's take a look at the results.

Analyzing the Results

Now that we have looked at the source code, let's analyze the result to see how our vanilla neural net performs.

```

1 losses = mlp.train(train_x, train_y, epochs=2000, lr=0.01)
2 print()
3
4 for x, y in zip(train_x[:10], train_y[:10]):
5     print(x, y)
6     print(f"      ypred: {mlp.forward(x)}")
7
8 plt.plot(losses)
9 plt.xlabel("epoch")
10 plt.ylabel("loss")
11 plt.show()
12
13 #mlp.show_params()

```

```

Epoch: 100/2000, Loss: 0.0006795237745380497
Epoch: 200/2000, Loss: 0.0002780199905385479
Epoch: 300/2000, Loss: 0.00015780133554953304
Epoch: 400/2000, Loss: 9.996884725589866e-05
Epoch: 500/2000, Loss: 6.544552166502369e-05
Epoch: 600/2000, Loss: 4.373246439583505e-05
Epoch: 700/2000, Loss: 3.032175895325618e-05
Epoch: 800/2000, Loss: 2.2295750007135315e-05
Epoch: 900/2000, Loss: 1.7559747007674843e-05
Epoch: 1000/2000, Loss: 1.4723606896641237e-05
Epoch: 1100/2000, Loss: 1.2951440585334302e-05
Epoch: 1200/2000, Loss: 1.1773900112740833e-05
Epoch: 1300/2000, Loss: 1.0936643742787753e-05
Epoch: 1400/2000, Loss: 1.0302935457567813e-05
Epoch: 1500/2000, Loss: 9.79811628836463e-06
Epoch: 1600/2000, Loss: 9.379957580934361e-06
Epoch: 1700/2000, Loss: 9.023360655331247e-06
Epoch: 1800/2000, Loss: 8.712547584218995e-06
Epoch: 1900/2000, Loss: 8.437042172730164e-06
Epoch: 2000/2000, Loss: 8.189547717119376e-06

```

```

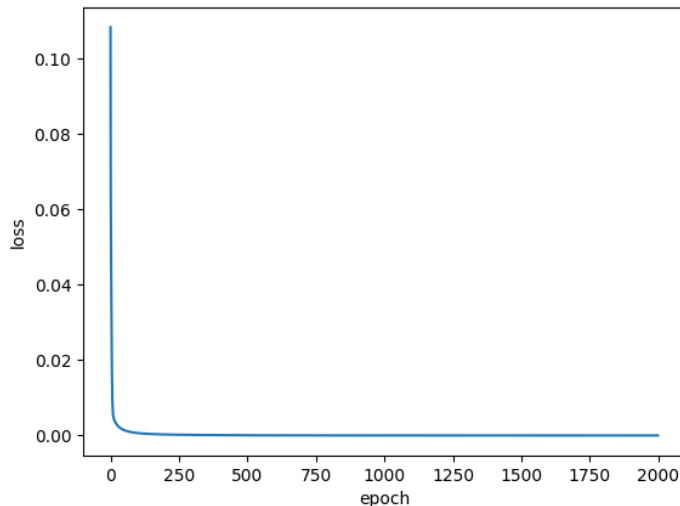
[1.0, -1.0] -1.0
      ypred: -0.9914314084230468
[0.8, -1.0] -0.8
      ypred: -0.8026975789984329
[0.8, -0.9] -0.7200000000000001
      ypred: -0.7229659139930072
[0.1, -0.8] -0.08000000000000002
      ypred: -0.07901899113208088
[0.3, -0.3] -0.09
      ypred: -0.08931568886644314
[-0.7, -0.1] 0.06999999999999999
      ypred: 0.07003157242570901

```

```

[-0.6, -0.5] 0.3
    ypred: 0.29863205953269367
[0.7, 0.8] 0.5599999999999999
    ypred: 0.5613061391881997
[-0.5, -0.4] 0.2
    ypred: 0.19629911113434118
[0.9, 0.8] 0.7200000000000001
    ypred: 0.7218073084017129

```



I would say this is pretty good! The loss is constantly decreasing and the prediction seems to be pretty good. Looking at the graph, the model seemed to have little idea at first, but instantly learned to see the holistic view as suggested by the sudden drop.

```

1 test_loss = 0
2 for x, y in zip(test_x, test_y):
3     ypred = mlp.forward(x)
4     test_loss += loss(y, ypred)
5
6 for x, y in zip(test_x[:10], test_y[:10]):
7     ypred = mlp.forward(x)
8     print(x, y)
9     print(f"    ypred: {ypred}")
10
11 print("Test Loss:", test_loss / len(test_x))

```

```

[-0.9, -0.3] 0.27
    ypred: 0.27224246623233583
[0.9, 1.0] 0.9
    ypred: 0.889162612694173
[0.1, 0.5] 0.05
    ypred: 0.050824340568330406
[1.0, 1.0] 1.0

```

```
ypred: 0.9737260427905209
[0.6, -0.8] -0.48
ypred: -0.4811987339447982
[0.6, -0.3] -0.18
ypred: -0.17587974783858806
[0.6, 0.4] 0.24
ypred: 0.23577905969192914
[0.5, -0.8] -0.4
ypred: -0.4007379739663608
[0.5, 0.2] 0.1
ypred: 0.0970149401772119
[0.9, -0.2] -0.18000000000000002
ypred: -0.18071247280108016
Test Loss: 1.1432408362631248e-05
```

Our test also seems pretty good. The loss seems small because our validation set uses small numbers, but we could see that the prediction somewhat matches the real values!

Now looking back at our model, this is again good for the limited range as suggested by the result above. However, there is a reason for that. Recall that we shuffled the dataset and selected 80% for training and 20% for testing there. Therefore, for the limited range that the model is trained in, the model did pretty good job. However, when I first tried making dataset by splitting the training and validation as first 80% and latter 20%, our model did poorly. This is because the model learned the left side well, but it does poorly in predicting the right side since it has no idea what's going on there with absolutely no training example. However, the good news is that our neural net is scalable unlike the previous example! By increasing the range for datasets, and could see whether the model underfits or overfits.

We now covered two models from scratch! Now that we know what happens in deep neural nets, let's build our own language models!

Building Language Models

From the third note, we discussed foundations and intuitions on language models. In our previous note, we discussed about in-depth about neural networks. Before going into different topics including AI safety, I wanted to build language models because there are a lot of concepts to discuss there. First, using our knowledge, let's build a bigram. Then, let's build a more sophisticated language model that actually sounds like a chatbot with the introduction of transformers.

§6.1 Bigram Language Model

§6.2 Attention and Transformers

Finally! Its time to discuss about transformers. If you have not already, please complete the playlist by 3Blue1Brown's highlighted in the last note and read [Attention Is All You Need](#). More great resources can be found in the last note. *Attention Is All You Need* is considered one of the most important paper in computer science and behind its meme-like title, it makes current LLMs possible.

Although the transformer architecture is used in many places, I want to introduce it LLMs as I think it is most intuitive and classic. The problem with prior language models is that they can't remember what they were talking about seconds ago. Because the next token is predicted with limited context window, it is not possible to "remember" the entire context like us. To improve this, eight researchers from Google introduced transformers. Although the basic ideas existed before the paper, it made the transformers well known.

The content itself, when you think about it, is quite simple and very intuitive. What should we do to enhance the use of information that we have? The answer is quite simple and circular. Literally use more prior contexts with efficient methods. Think of transformer blocks, or layers, as a function that takes in and spits out vectors of the same shape. The input vector goes in

to a magical concept called multi-head attention, add and norm, and feed forward (which is basically a fancier word for MLPs that we discussed so far).

§6.3 Building an LLM with PyTorch

Note **7**

AI and The Future

§7.1 AI Safety

Further Resources

Throughout learning, I was blessed with countless resources dedicated for machine learning. Therefore, I decided to list recommended readings and other resources I used when studying and writing the notes. A huge shout out to creators for their contributions in machine learning community!

Papers

- **All Time Favorites**
 - [Attention Is All You Need](#)
 - [A Mathematical Framework for Transformer Circuits](#)
- **LLMs**
 - [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#)
 - [Chain-of-Thought Reasoning Without Prompting](#)
 - [Language Models are Few-Shot Learners](#)
 - [Scaling Laws for Neural Language Models](#)
 - [A Neural Probabilistic Language Model](#)
 - [Large Language Models: A Survey](#)
- **Diffusion Models**
 - [Hierarchical Text-Conditional Image Generation with CLIP Latents](#)
 - [Score-Based Generative Modeling through Stochastic Differential Equations](#)
 - [Denoising Diffusion Probabilistic Models](#)
 - [Learning Transferable Visual Models From Natural Language Supervision](#)

Websites

- **Articles and Blogs**
 - [Colah's Blog](#) (Christopher Olah)
 - [When AI builds itself](#) (Anthropic)
 - [Deep Double Descent](#) (OpenAI)

- [Visualizing K-Means Clustering](#) (Naftali Harris)
- [Jason Osajima](#) (Jason Osajima)
- [ML Foundations: Understanding the Math Behind Backpropagation](#) (Jordan Coblin)
- **Miscellaneous**
 - [MLU-EXPLAIN](#) (Machine Learning University)
 - [FrontierMath](#)
 - [Machine Learning Mastery](#)
 - [xKiwiLabs](#)
 - [Distill](#)
 - [PyTorch Tutorial](#)
- **GitHub**
 - [ML-From-Scratch](#) (Erik Linder-Norén)

Books and Notes

- **Books**
 - [Goodfellow's Book](#)
 - [Dive into Deep Learning](#)
 - [Neural Networks and Deep Learning](#) (Michael Nielsen)
 - [Mathematics for Machine Learning](#)
 - [Deep Learning with PyTorch](#)
 - [Foundations of Computer Vision](#) (Antonio Torralba, Phillip Isola, William Freeman)
- **Notes**
 - [CS229](#) (Stanford University)
 - [Sparse Autoencoder](#) (Andrew Ng)

Courses and Videos

- **Courses**
 - [Practical Deep Learning](#)
 - [CME 295](#) (Stanford University)
- **Videos**
 - [Neural Networks](#) (3Blue1Brown)
 - [Neural Networks: Zero to Hero](#) (Andrej Karpathy)

Tools and Datasets

- **Tools**
 - [Weights & Biases](#)
 - [OpenRouter](#)
- **Datasets**
- **Platforms**
 - [Cirleback](#)
 - [Fireflies](#)